

Project Portfolio

Noah Hester

PHYS/EMEC 310: Electronics

Dr. Brian Rasnow

May 12, 2025

Table of Contents

1. DC Power Supply	3
Build an adjustable DC power supply (that's linear with potentiometer R2).	
a. Methods	3
b. Results	4
c. Discussion	7
d. Conclusion	7
2. Transistor Gain	8
Measure the current gain, $i_c/i_b = \beta = h_{fe}$, of an NPN transistor for a few different collector resistors.	
a. Methods	8
b. Results	9
c. Discussion	13
d. Conclusion	13
e. Appendix	13
3. Oscilloscope Lab	15
Construct an apparatus to visualize fast time-varying voltages (what an oscilloscope does) and also display spectra (what a spectrum analyzer does).	
a. Methods	15
b. Results	17
c. Discussion	23
d. Conclusion	24
e. Appendix	24
4. Arduino Controlled DC Power Supply	28
Design, build, and test an Arduino controlled LM317 power supply.	
a. Methods	28
b. Results	31
c. Discussion	36
d. Conclusion	36
e. Appendix	36
5. Arduino Controlled Function Generator	
Design, build, and test an Arduino controlled function generator.	
a. Methods	39
b. Results	42
c. Discussion	46
d. Conclusion	46
e. Appendix	46

DC Power Supply

Objective: Build an adjustable DC power supply (that's linear with potentiometer R2).

Methods:

1. Identify the pins of the LM317 integrated circuit, and model the output voltage. We first do this theoretically with a model in MATLAB based on the datasheet for the component. This datasheet describes this equation: $V_o = V_{REF}(1 + R2/R1) + I_{ADJ}R2$

Which corresponds to the following diagram for a basic adjustable voltage regulator:

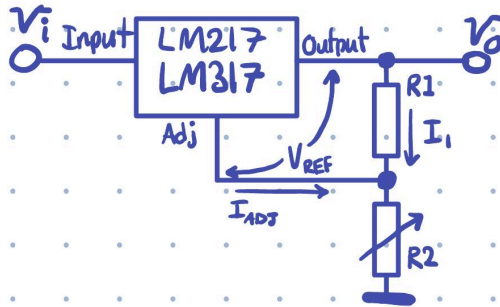


Figure 1: Schematic for basic adjustable voltage regulator in the LM317 data sheet.

For our data model, the $I_{ADJ}R2$ term was ignored since the datasheet says it may often be neglected, which helped simplify the equation. A model was then created in MATLAB to yield a theoretical graph for voltage versus resistance.

However, this model has a major flaw in that it is inaccurate above ~11 V since the circuit cannot output more than its input. So a second model was constructed that constrains the voltage to the correct range, making the curve flat when it reaches the maximum voltage.

2. An ideal resistance was then calculated to achieve the desired voltage range. This can be done by solving the simplified version of the equation from above, $V_o = V_{REF}(1 + R2/R1)$, for the fixed

$$\text{resistance } R1: R1 = \frac{V_{REF}R2}{V_o - V_{REF}}$$

This resistance should give us the desired voltage range from 1.25V to 11V.

3. Create a working prototype of our adjustable voltage supply on a solderless breadboard. A prototype was created on a breadboard using the closest resistance value to the theoretical one calculated before. This was done to ensure the circuit outputted the expected minimum and maximum voltage values. The circuit followed this schematic:

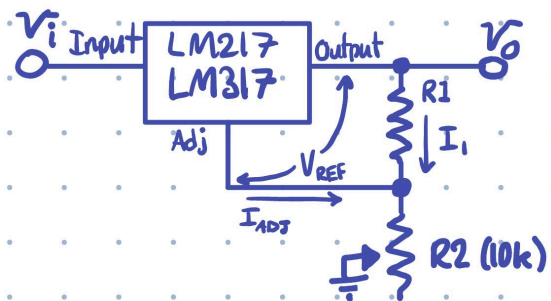


Figure 2: Schematic for prototyped circuit.

4. Upon success of the prototype, the circuit was recreated on the soldered breadboard. For the final circuit, a few other components were added, including a capacitor, a 3-digit voltmeter, and some LEDs for feet. In addition, a single component was used to function as both a potentiometer and a switch. The final circuit diagram appears as so:

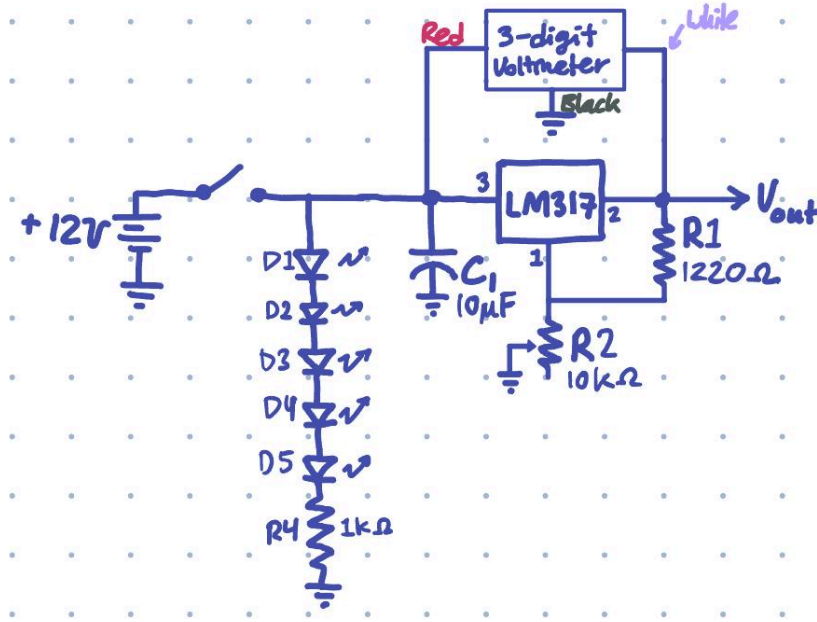


Figure 3: Schematic for final circuit

5. Testing, debugging, and validating the circuit. The circuit was tested to see whether the output voltages matched the expected values, as well as if the 3-digit voltmeter value is comparable to the digital multimeter value. The $R_{Thevenin}$ was also measured and noted.

Results:

1. The first model of the adjustable voltage supply, implementing just the simplified equation, gave a linear curve when plotting output voltage versus resistance (R2):

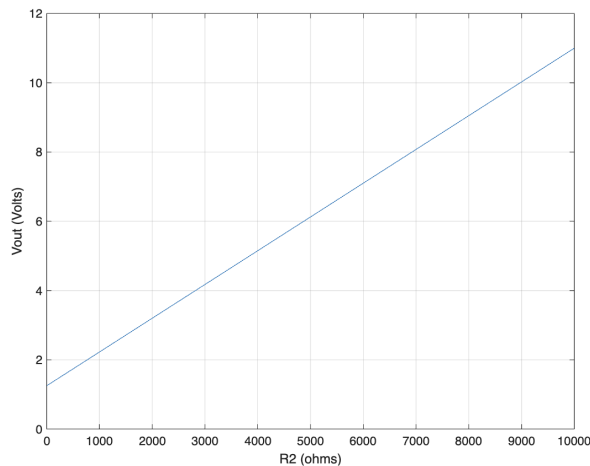


Figure 4: First voltage plot from LM317

The second, more accurate model created a similar curve, except flattened at the top where the voltage hits maximum:

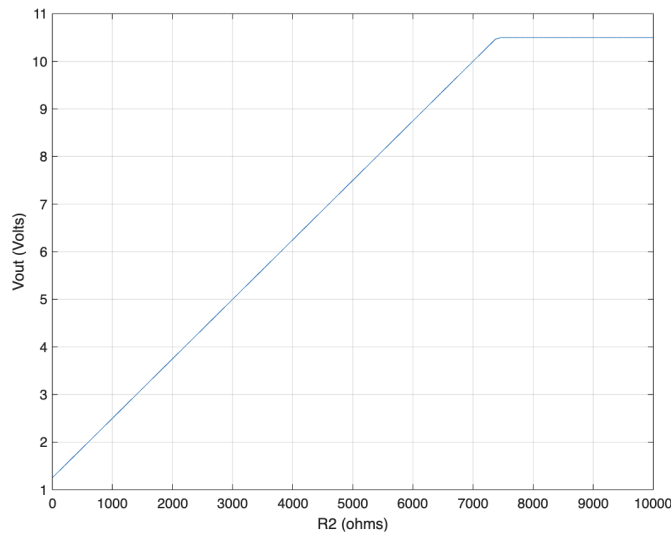


Figure 5: Voltage output from LM317 after adjustment.

- At our desired maximum output voltage of 11 volts, our potentiometer will be at its highest value of 10k Ω . Our V_{REF} will be 1.25 volts as this is stated to be a typical value in the data sheet for the component (LM317). Seeing as we have eliminated all unknowns except for R1, we can now calculate it with the formula: $R1 = \frac{(1.25V)(10000\Omega)}{11V - 1.25V} = 1282\Omega$
In practice, a 1k Ω and a 220 Ω resistor were used in series, as the combined voltage would come close to this value.

- The circuit built on the breadboard also included a green LED to give a sense for the output voltage:

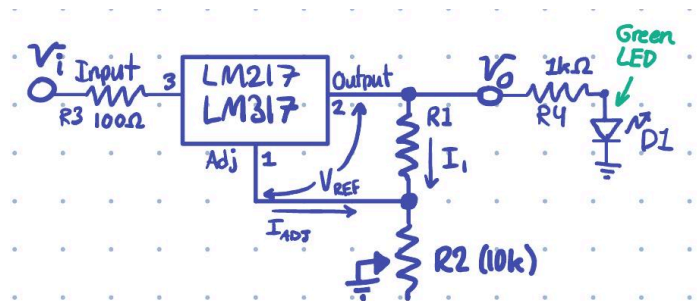
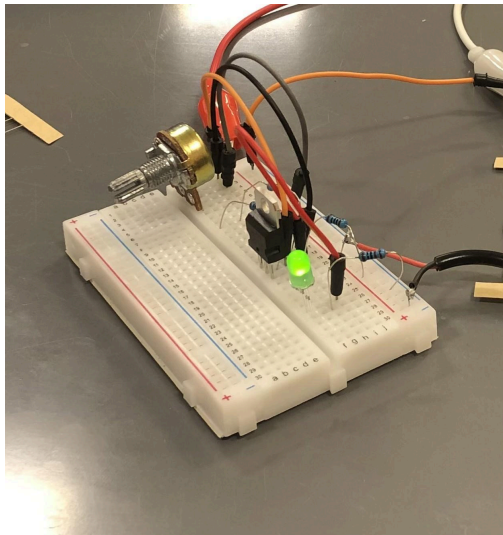


Figure 6: Photo of the breadboard prototype. **Figure 7:** Schematic for prototype including LED.

For this particular circuit, $1k\Omega$ and 330Ω resistors were used for R1, also close to the target resistance, and the measured value for this combined resistance was 1329Ω . The maximum resistance of the potentiometer was 9880Ω , and the measured input voltage was $12.15V$. The measured output voltage was $10.9V$, which is a good sign as it is very close to $11V$, and the measured reference voltage was about $1.181V$, which is near to the theoretical $1.25V$.

4. Seeing as the previous step was successful and displayed voltages near to what was expected, the final circuit was then created and soldered into place, however, the 330Ω resistor was substituted for a 220Ω resistor in order to slightly increase the voltage to better meet the specifications.

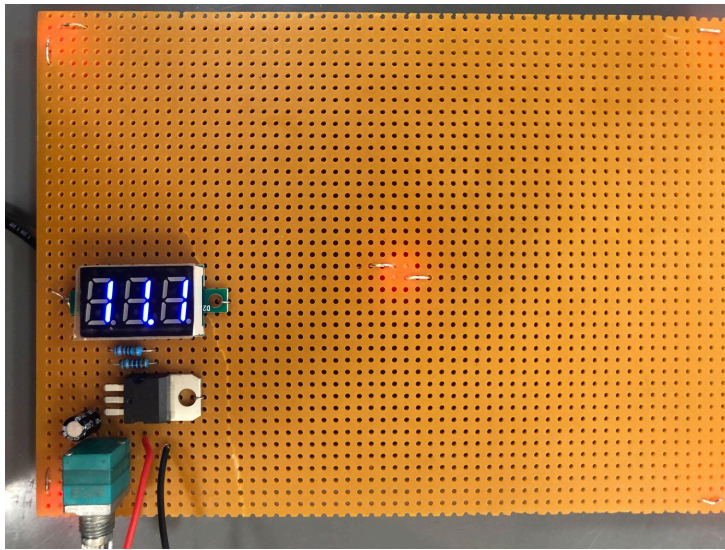


Figure 8: Full front side of board

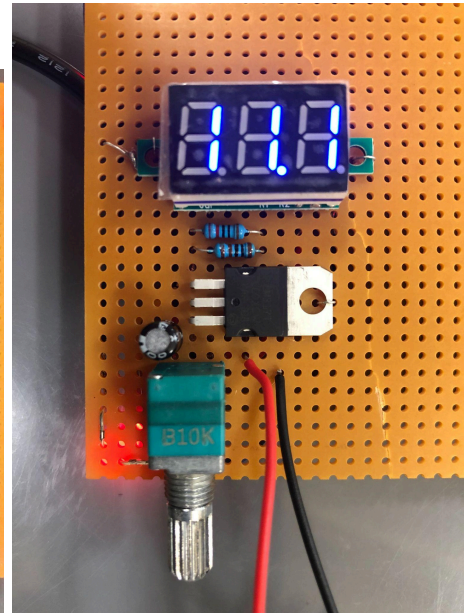


Figure 9: Closer look at the front of circuit.

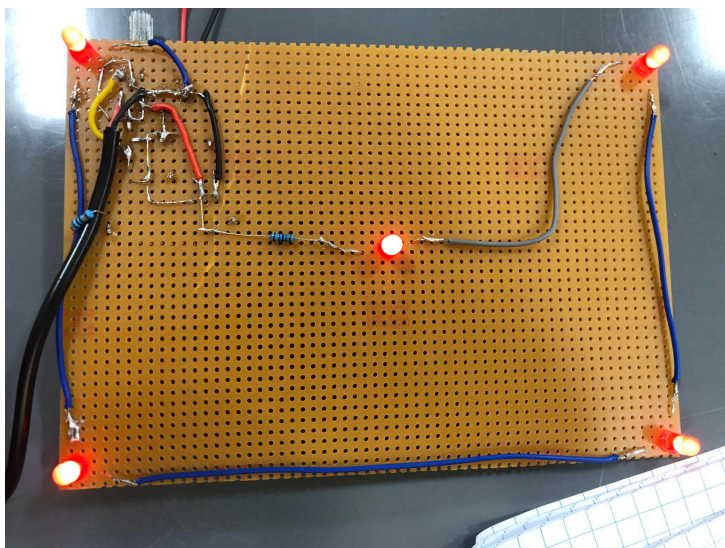


Figure 10: Full back of board.

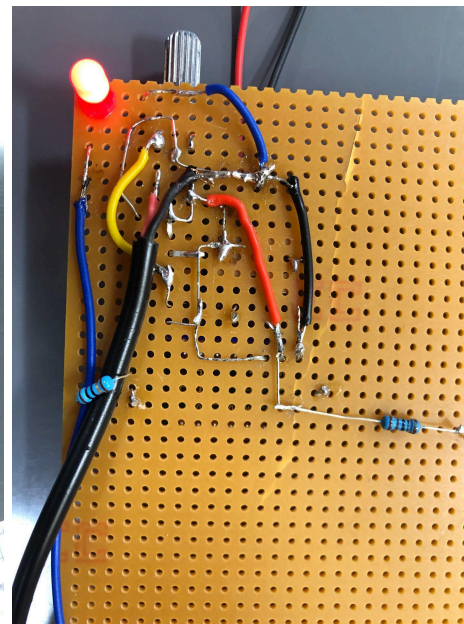


Figure 11: Closer look at the back of circuit.

5. After successful construction of the voltage supply, it was subject to testing in order to see whether the output voltages matched expectations, how accurate the 3-digit display is, and to discover the Thevenin resistance of the voltage supply.

The minimum measured voltage output was 1.258V with the multimeter, and the 3-digit display showed 1.25V (good accuracy).

The maximum measured voltage output was 11.25V with the multimeter, and the 3-digit display showed 11.1V (near to actual).

For specifications, the maximum resistance of the potentiometer was measured to be 9220Ω, and the minimum was 2.2Ω. The combined resistance of the 1kΩ and 220Ω resistors (R1) was measured to be 1206Ω.

The reference voltage at maximum was measured to be 1.184V, which is relatively close to the theoretical value of 1.25V.

The Thevenin voltage of the supply was measured as 11.33V at maximum, and, after a 100Ω load was placed across it, the voltage dropped to 11.11V. The Thevenin resistance of the supply was then calculated using the formula $R_{th} = R_{load} \left(\frac{V_{th}}{V_{load}} - 1 \right)$ to be 1.97Ω. This was derived from the voltage divider equation by considering the power supply to be a Thevenin voltage and resistance in series with a load, giving: $V_{load} = V_{th} \left(\frac{R_{load}}{R_{load} + R_{th}} \right)$. This can then be algebraically solved for R_{th} in order to get the above equation.

Discussion:

The MATLAB model in code allowed us to save a lot of trial, error, and computation time in that it allowed us to compute the correct theoretical resistance for our final circuit without having to actually build it. The data sheet for the LM317 component was also extremely helpful as it gave us the equation for the output voltage, and also allowed us to simplify our calculations for our models.

In the final circuit, there was also a capacitor added into the circuit in order to stabilize the voltage supply, since the capacitor can help eliminate fluctuations by storing charge that will assist in preventing small voltage variations. The specific capacitor added into the circuit has a capacitance of 10μF.

The five LEDs that function as mechanical stabilizers (legs) for the board, were wired in series to the voltage source in addition to a 1kΩ resistor that would ensure the proper current, about 20mA, for the LEDs that meets the specifications. A red LED has a typical ideal voltage of around 1.8V and a current of about 20mA or 0.02A, and so we can compute the required resistance to achieve this using Ohm's Law. We know we need to step down the voltage by 12.15V (measured) - 1.8V = 10.35V across the resistor, and so we can calculate $R = V/I = 10.35V/0.02A = 517.5\Omega$. And so we can see that a 1kΩ resistor would be more than enough resistance to maintain a safe current for the LEDs.

Conclusion:

This lab was a good exercise exploring how the LM317 integrated circuit can be used to create a much better and more reliable voltage supply for future projects. It also gave me good experience with soldering, helping me get better at bonding components together.

Transistor Gain

Objective: Measure the current gain, $i_c/i_b = \beta = h_{fe}$, of an NPN transistor for a few different collector resistors.

Methods:

1. Design a circuit that will allow us to compute beta. To calculate beta, we need both the transistor base current and collector current so that we can compute it using the formula for an active transistor, $\beta = I_c / I_b$. Now, it can be hard or potentially dangerous to directly measure current, so a circuit was created to instead measure voltages coming from three points on the circuit—the voltage before and after R4 (Fig. 1), and between the transistor and R1 (Fig. 1). The resistors are placed in this way in order to allow us to calculate the voltage difference across them, and then compute current later by Ohm's Law. So these three voltages, as well as the reference voltage from the arduino, allow us to compute the current through R1 (Fig. 1), which is also the transistor's collector current, and the current through R4 (Fig. 1), which is also the transistor's base current.

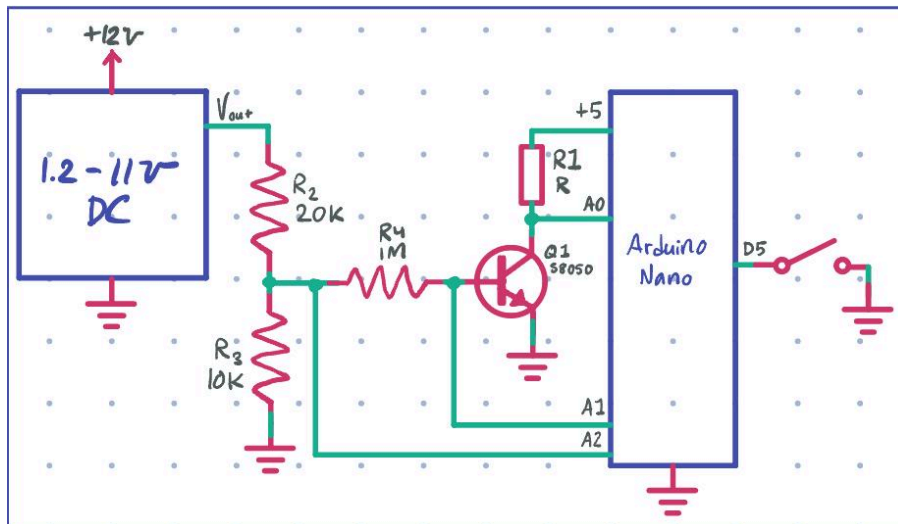


Figure 1

2. Creating the actual physical circuit. No 20k Ω resistors were available when building the circuit, and so two 10k Ω resistors, labeled R2a and R2b respectively (Fig. 2), were used instead to create the actual circuit, whose modified layout can be seen in Figure 2.

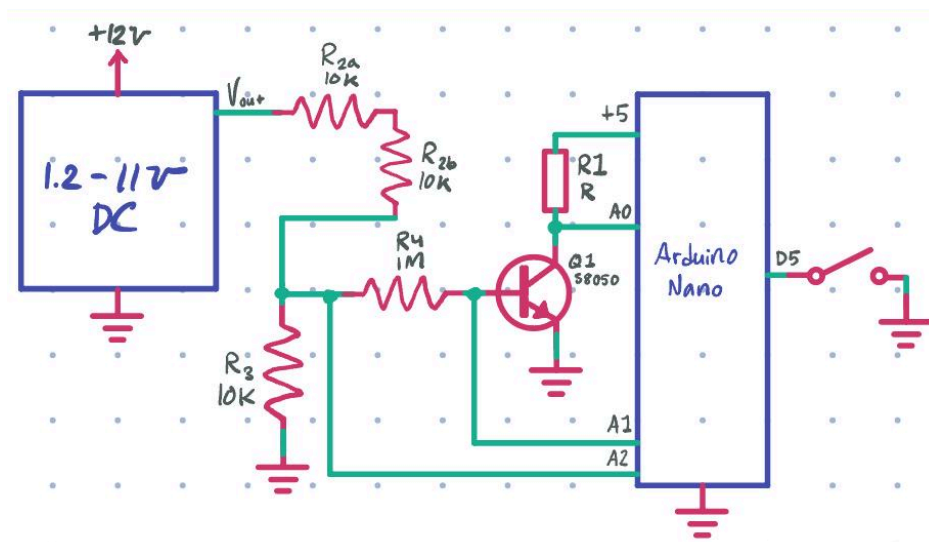


Figure 2

3. Using an Arduino Nano to measure voltages. Using a microcontroller to measure these voltages allows us to measure a lot of data very quickly without having to go back and forth with a digital multimeter. Getting a larger amount of data for a range of input voltages V_{out} allows us to get more accurate results for our calculated values. A number of different resistors were used for R1 (Fig. 2), including 1k Ω , 5.1k Ω , and 10k Ω in order to measure beta for different collector resistances and to observe transistor behavior for different R1 values.

4. Calculating currents and the transistor beta value. Once we have the voltages, we can then compute the current through the collector and base of the transistor by calculating the voltage differences across the resistors using: $V_{R4} = (V_{A2} - V_{A1})$, $V_{R1} = (V_{ref} - V_{A0})$

(Note: V_{A2} is the input voltage, V_{A1} is the base-emitter voltage, and V_{A0} is the collector-emitter voltage)

When we get these calculated voltages, we can then calculate the desired currents using:

$$i_b = V_{R4} / R4, i_c = V_{R1} / R1$$

For this purpose, all the resistances in the circuit were measured to acquire actual values that will give more accurate results for our beta value. Once the currents are computed, we can easily compute beta as so: $\beta = i_c / i_b$

Results:

1 & 2. The final circuit is as shown in figure 3. It was built on a breadboard and the resistor R1 was switched out for multiple measurement with different resistances.

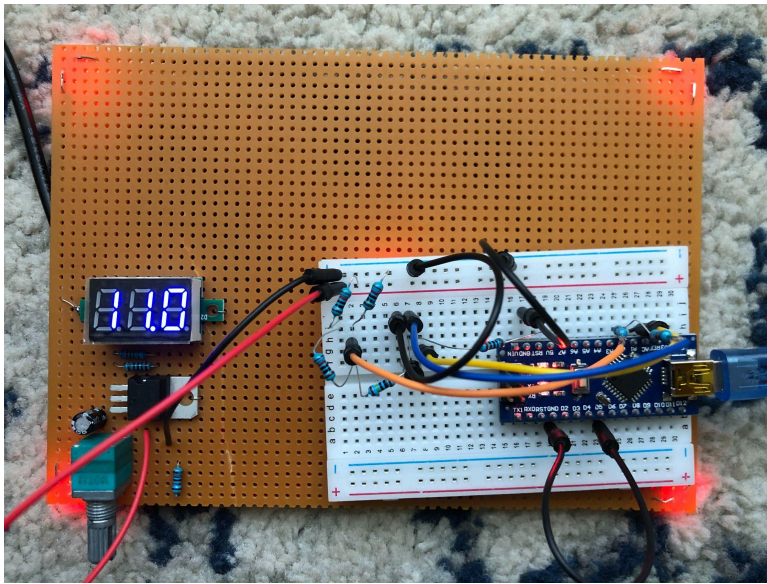


Figure 3

Figure 4 shows a closer view with labeled components to show the circuit setup in actuality. The voltage is supplied by an LM317 power source as shown in Figure 3.

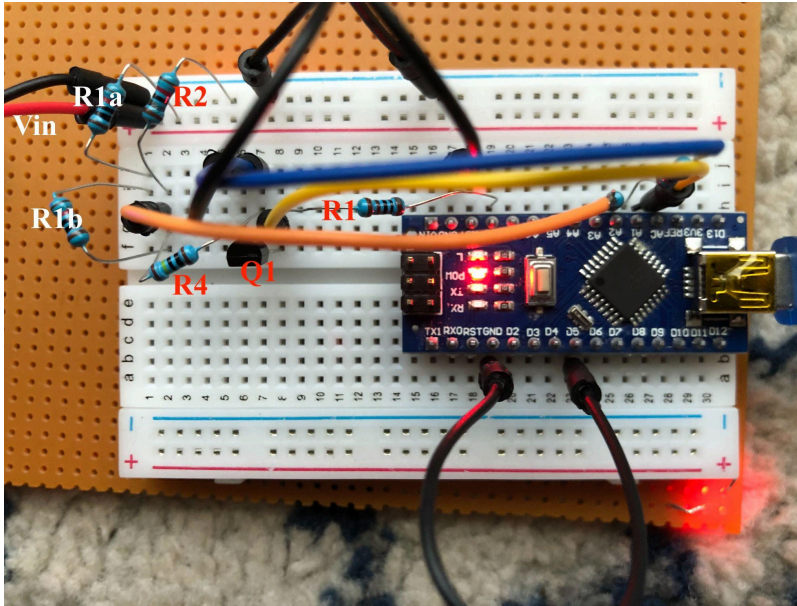


Figure 4

3. The Arduino Nano measured the three voltages running a simple analog reading program (shown in the Appendix), and this process was repeated for three different resistance values for R1. Here is an example of the graph of the voltages for $R1 = 1k\Omega$ (Fig. 5):

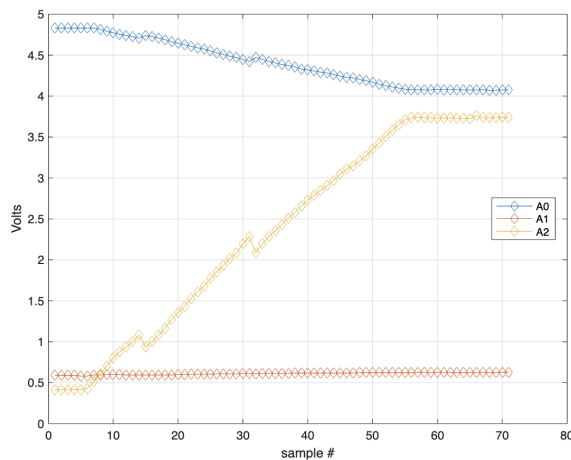


Figure 5

4. This voltage data was then taken and the resistance values were measured. The measured values were $R2 = 19.8k\Omega$, $R3 = 9.96k\Omega$, and $R4 = 0.947 M\Omega$. R1 was measured for each case (1k, 5.1k, and 10k), and these measured values were 1005Ω , $5.11k\Omega$, and $9.81k\Omega$. These were used to calculate the base and collector currents which were graphed (Fig. 6-8) to show their linear relationship (whose slope is beta):

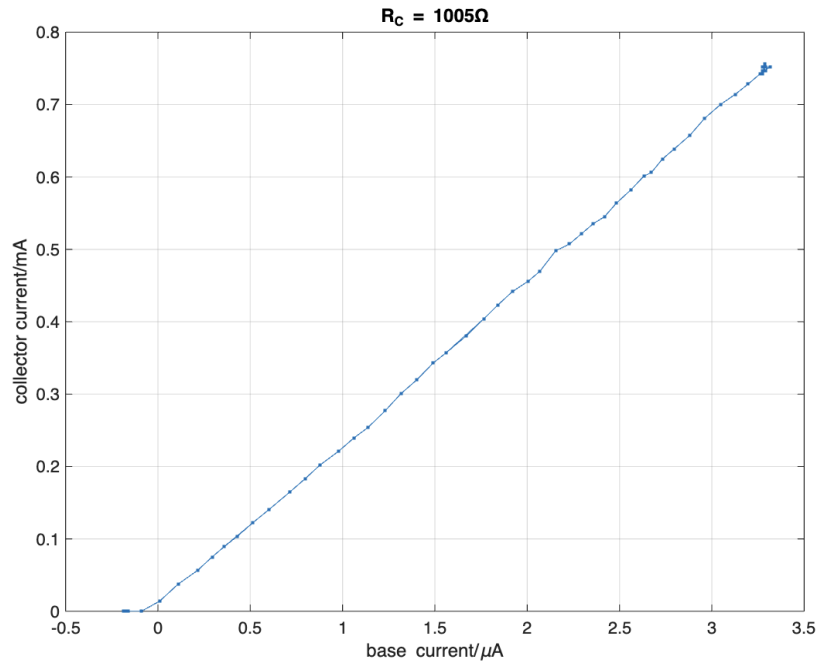


Figure 6

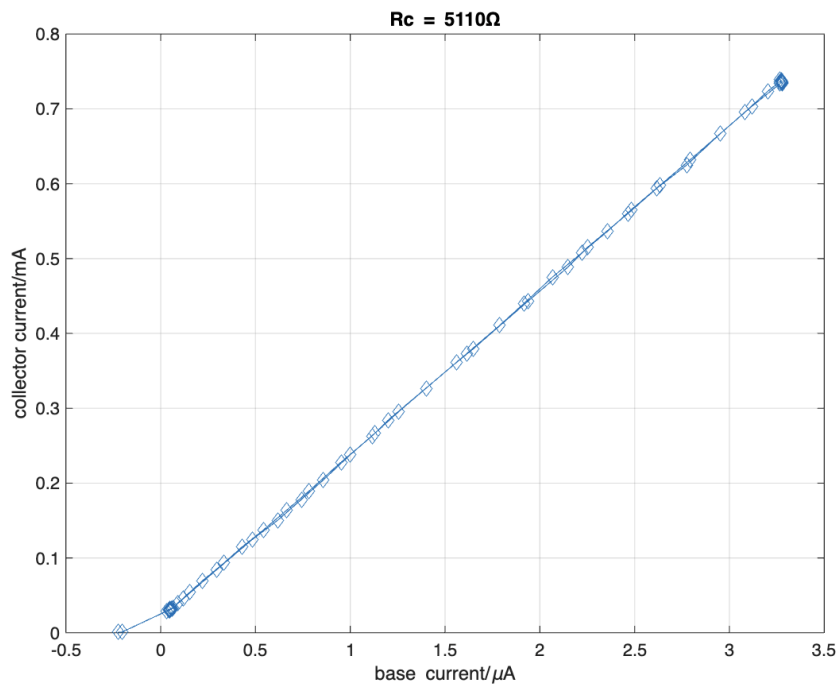


Figure 7

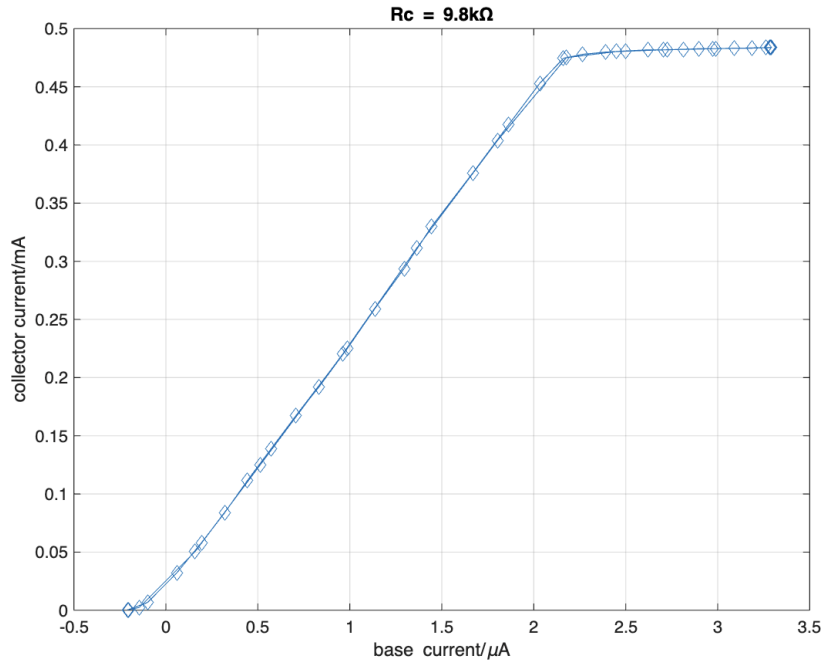


Figure 8

Figure 6 shows the graph for $R_1 = 1\text{k}\Omega$, Fig. 7 shows it for $R_1 = 5.1\text{k}\Omega$, and Fig. 8 shows it for $R_1 = 10\text{k}\Omega$.

4. Now, we can calculate beta by approximating the slope of these lines. We can do this using the MATLAB `polyfit()` function, and we get the following beta values:

$R_1 = 1\text{k}\Omega$ gives $\beta = 226.0$

$R_1 = 5.1\text{k}\Omega$ gives $\beta = 219.8$

$R_1 = 10\text{k}\Omega$ gives $\beta = 216.0$

We can see all these values are pretty close to each other, but if we graph them together (Figure 9), we can get a visual representation of their similarity as well:

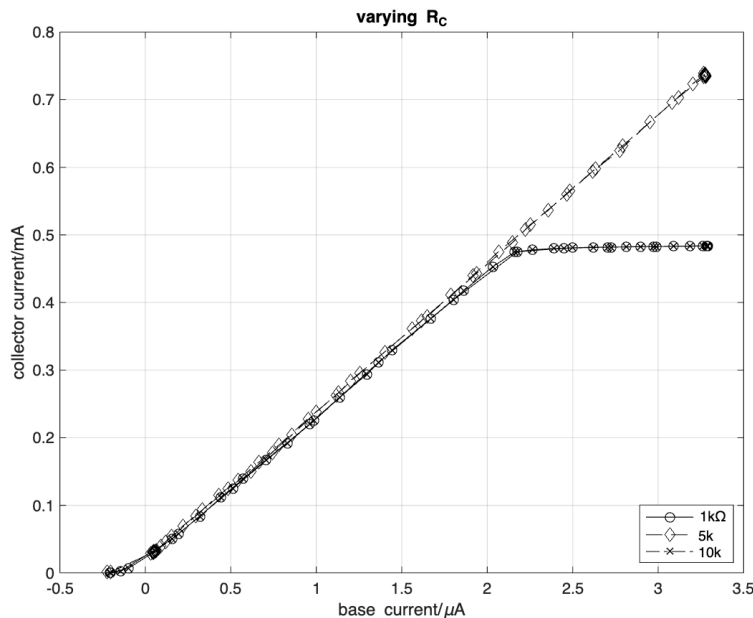


Figure 9

Taking this a step further, we can use MATLAB to calculate the mean and standard deviation to get: $\beta = 217 \pm 2$

Discussion:

I thought this was a very interesting lab both in approach and execution. I thought it was fascinating that we could measure the currents by putting known resistances on the transistor and measuring those voltages instead. This provides a genius solution that allows us to indirectly measure current and use it to calculate beta.

From the data results, we can see that the relationship between the base current and collector current is definitely linear, but in Fig. 8 we see something interesting—the current graph flattens at the top, reaching a constant value. This is due to saturation of the transistor, and provides an interesting example of what happens when the transistor reaches its saturated state. I thought it was nice to actually visually see the linear relationship between the currents in active state, and also see how it flattens out as the transistor becomes saturated—I think it gives some more understanding an intuition for it than simply looking at equations, so I found that to be helpful for deepening my understanding of transistors.

Using a microcontroller for automated voltage measurements was also extremely useful as it allowed for greater quantities of measurements with less effort since it did require manually measuring with a digital multimeter. It also shows the power of microcontrollers and how quickly they can perform tasks like this in comparison with manual methods.

Conclusion:

This lab was a valuable exercise in learning how to approach a problem like this, compose a circuit for accomplishing the task, and using automated measurements to increase productivity and accuracy. It also gave me some good experience translating circuit diagrams to physical circuits and understanding how to do that effectively.

Appendix:

Arduino Nano program for measuring voltages:

```
void setup() {
    // put your setup code here, to run once:
    Serial.begin(9600);
    pinMode(5, INPUT_PULLUP);
    analogRead(A0);
}

void loop() {
    // put your main code here, to run repeatedly:
    if(!digitalRead(5)) {
        Serial.print(analogRead(A0));
        Serial.print("\t");
        Serial.print(analogRead(A1));
        Serial.print("\t");
        Serial.println(analogRead(A2));
    }
}
```

```
    delay(200);  
}
```

Oscilloscope Lab

Objective: Construct an apparatus to visualize fast time-varying voltages (what an oscilloscope does) and also display spectra (what a spectrum analyzer does).

Methods:

1. **Hardware Interface:** Figure 1 shows the schematic for the first hardware interface that was constructed with the Arduino to read raw voltages from the electret microphone. This circuit divides the full 5V of source voltage in order to operate the microphone at the specified range in the datasheet, which is 3V (standard) up to 10V (max.), and also limits the current to the desired range, again specified by the data sheet to be less than or equal to 5mA (max.).

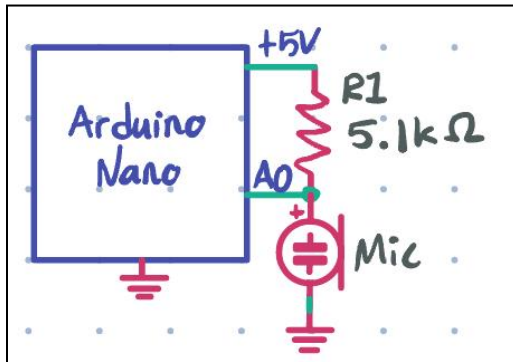


Figure 1

Measurements were made on this circuit using the software and data interfaces and varying levels of sound from different distances, etc. Upon completion of measurements from this circuit, a new circuit was created in order to amplify the small signal outputted by the microphone. In Figure 2, we can see by the resistor configuration that the circuit is set up with an inverting amplifier.

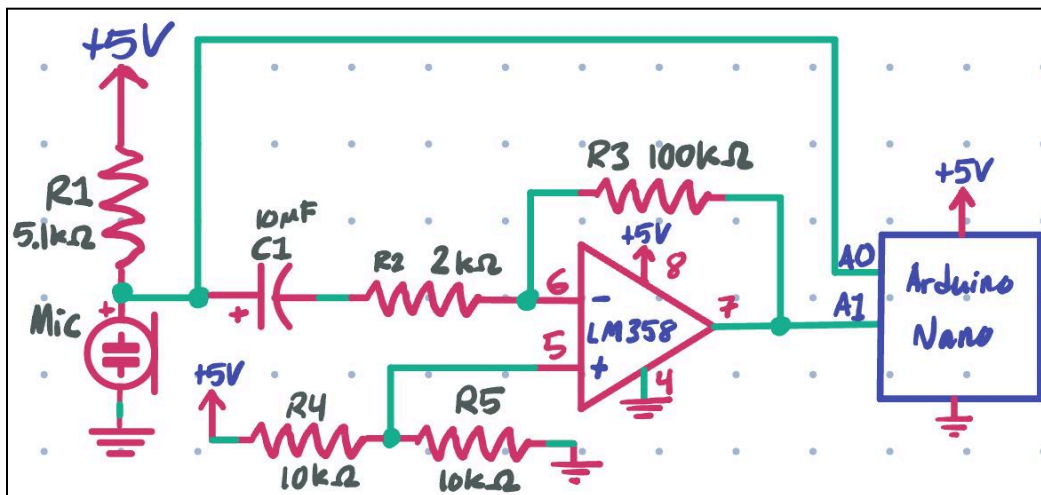


Figure 2

This inverting amplifier is set up with two resistors, $R_2 = 2k\Omega$, and $R_2 = 100k\Omega$. Since this is an inverting amplifier, we know that the gain of the amplifier is equal to:

$$- R_3/R_2 = - 100k\Omega/2k\Omega = (- 50)$$

So the purpose of the circuit is to amplify the signal from the microphone by 50 in order to better saturate the range on the Arduino so we can see and analyze the voltage better. We can also note that, by the configuration of resistors on the other terminal of the amplifier, this seems to be more specifically, a Difference Amplifier. This gain value that we calculated is still correct, but it means that it is amplifying not just the original microphone signal, which includes some DC offset that would overstate the measurement, but rather only the AC voltage which we then add a 2.5V DC bias to. We can also see we additionally measure the original raw microphone output we measured in the previous circuit for comparison. In this circuit, we also measured these values again after swapping the 10 μ F capacitor with a 0.1 μ F ceramic capacitor in order to see the effect upon the recorded frequencies.

2. Software Interface: For this project, we needed a piece of software to interface between our hardware and data analysis platforms, and so the Arduino runs the “oscope1.ino” program, written in C++, and given in the Appendix. The program implements a simple serial command interface with the Arduino, allowing us to request data or set the pulse width modulation value of pin D5. In this lab, we use only the measurement command, and we use it to quickly read many measurements from the Arduino every second, automating the measurement process and speeding it up. When the ‘b’ command is sent via serial connection, the arduino sends back the two analog pin measurements in binary through the serial port, as well as some timing measurements for calculating the sample rate. This allows for communication between the hardware and the program we will be using to graph and analyze the data created by the hardware.
3. Data Interface: The first iteration of this was “oscope1.m” (see the Appendix), which essentially connected to the Arduino via serial port, then repeatedly takes measurements and plots them for as long as a certain button is toggled by the user. In this way, it displays real time voltage data from the Arduino.

The second iteration, “oscope2.m” (Appendix), improved upon this concept in a couple ways, having error handling and also giving a framerate reading on the plot. It also gives time in milliseconds rather than microseconds, which is how the Arduino reads it. A trigger for automation was also added in the form of a flag variable, ‘runOnce’. If set to true, then it will run the oscope2 loop a single time.

These two programs were used to take voltage measurements from the aforementioned circuits in order to see the effect of different distances and/or volumes of sound upon the output, and to compare the raw output with the amplified output.

Another program of relevance to the data interface was “aspectrum.m” (see Appendix). This MATLAB program is a function that takes a data vector and sample rate, and performs the Fast Fourier Transform (FFT) in order to plot the data as it appears in the frequency domain. It does this using MATLAB’s “fft” function, then performs some index algebra to normalize the amplitude to ADU units and convert the frequencies to Hz. The output of this is a graph that contains spikes corresponding to the frequencies present within the data through the Fourier transform.

Results:

1. Figure 3 shows the physical construction of the circuit given by the diagram in Figure 1.

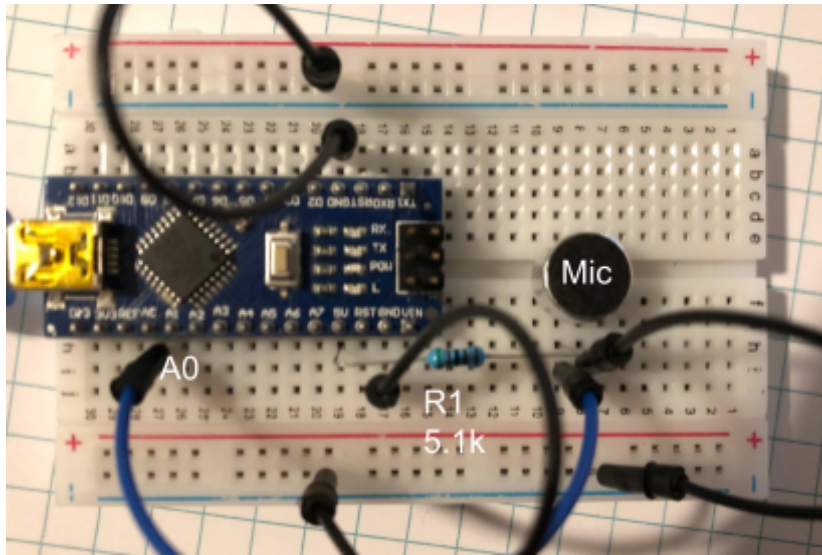


Figure 3

Figure 4 shows the more complicated constructed circuit with the OpAmp.

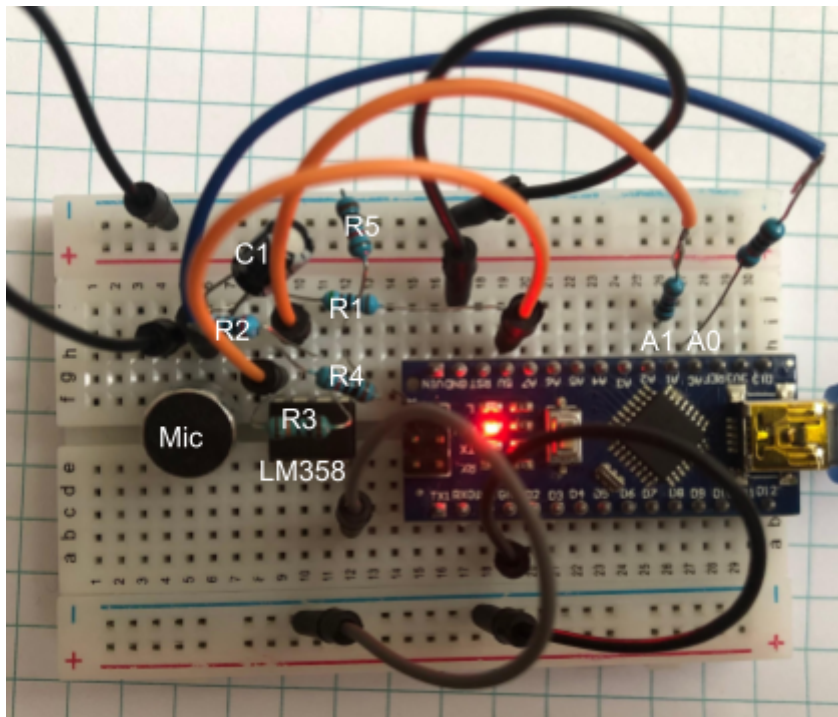
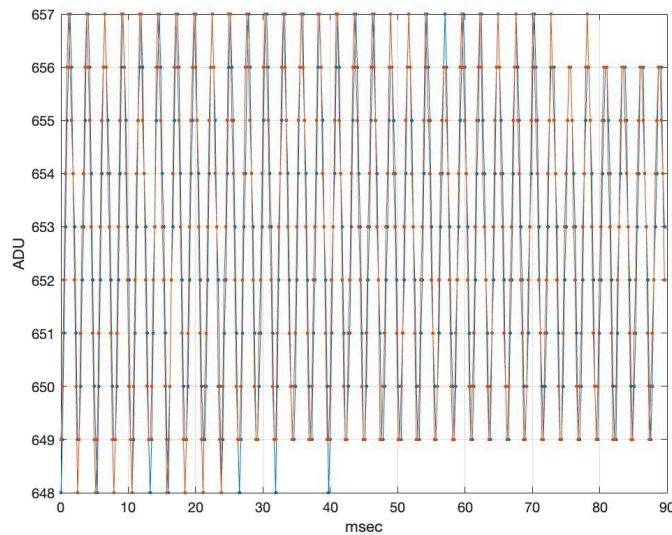


Figure 4

2. Figure 5 shows the output from the oscilloscope for the first circuit, using the raw data from the microphone (inputted into both A0 and A1 to keep the graph more visible).



This shows that the raw output is very small—the amplitude range goes from 648 to 657, a difference of only 9 ADU. This limits how much we can view this output because the arduino discretises it to the point that it is harder to analyze on this level. Hence the need for an amplifier to let us smooth that out and get better data to work with.

Figure 6 shows a sample output from oscscope2.m while humming a sound near the microphone, in the second amplifying circuit several inches away. For reference, the blue line is the raw microphone output from A0 on the Arduino, and the orange line is the output from the OpAmp fed into A1 on the microcontroller.

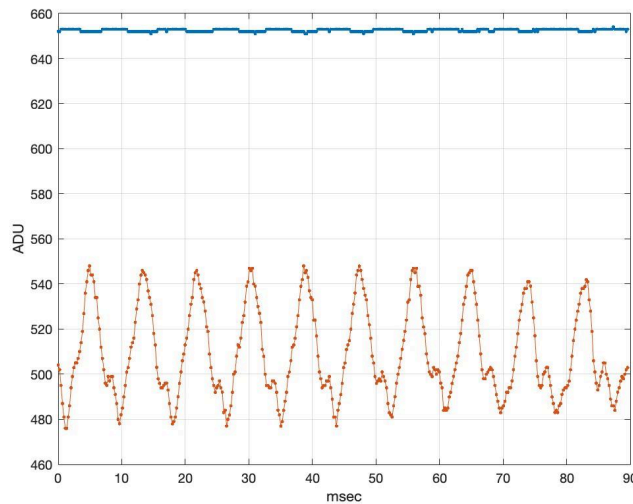


Figure 6

Figure 7 shows the matching frequency domain graph after the Fourier transform is performed on the data.

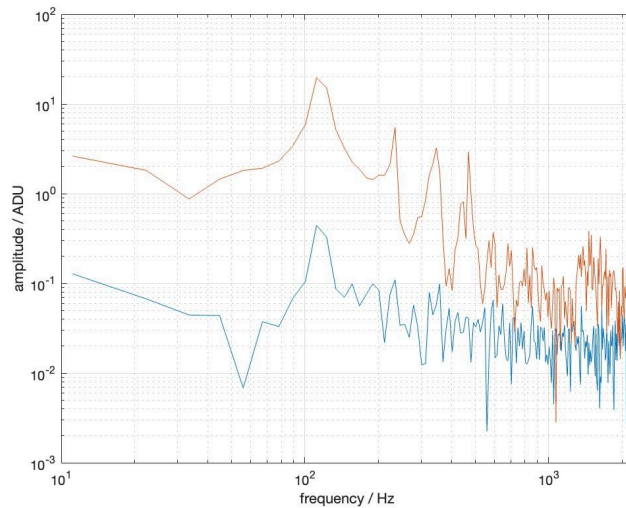


Figure 7

This frequency data has a number of larger spikes near the middle, in the range between 10^2 and 10^3 . This means that the sound that was produced had dominant frequencies in the 100s range according to this graph. Moving closer or farther away or changing the volume of the sound changed the amplitude of the voltage graph, but did not affect the frequency graph very much, while changing pitch and humming lower or higher notes (different frequencies), changed how many peaks were displayed on the voltage graph and shifted the spikes in the Fourier transform, showing that the sound frequencies (as represented in the voltages) were shifting up and down. Figures 8 and 9 show a higher frequency higher amplitude note that was being hummed, and we can see that in the data since the amplitude of the voltage has drastically increased and there is a large spike in the frequency at ~ 468 Hz, which is the dominant frequency that I was humming.

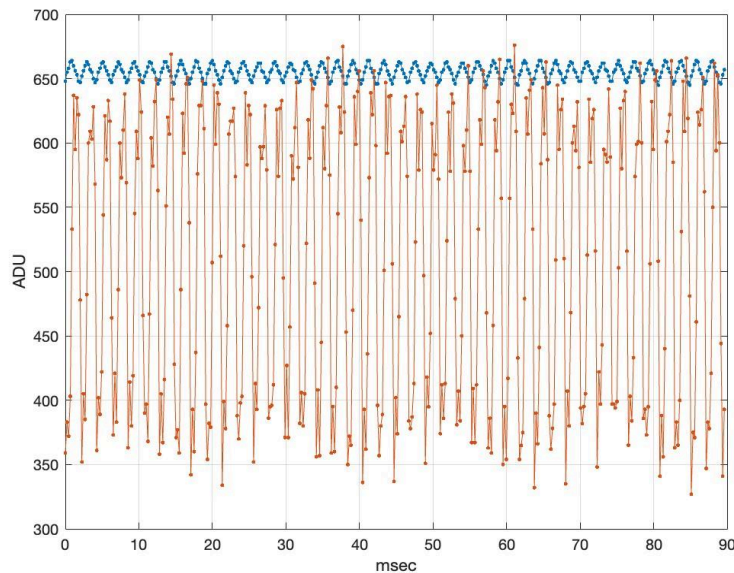


Figure 8

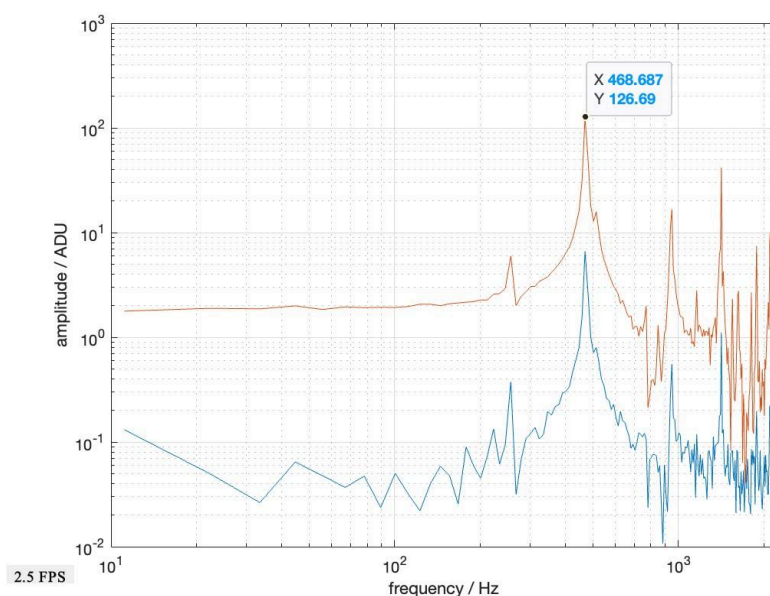


Figure 9

Now, looking back at Figure 8, we can see that there is a bit of drifting in the orange voltage at the bottom, which we can also see in Figure 10. This is a version of the circuit where C1 has a capacitance of $10\mu\text{F}$.

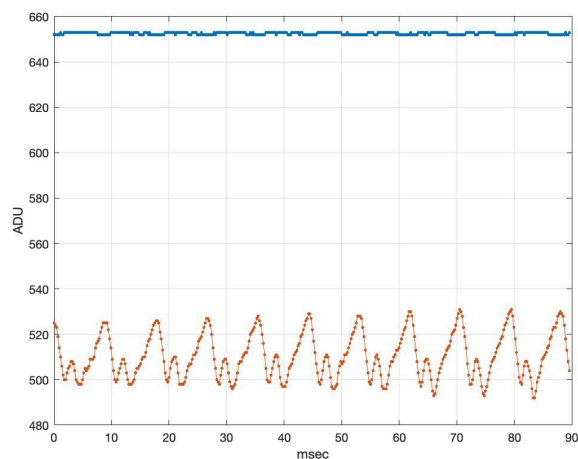


Figure 10

In contrast, figure 11 shows the same circuit with C1 being instead a ceramic capacitor with a value of $0.1\mu\text{F}$. This change helps filter out the ambient lower background frequencies and flattens the voltage. Though the change is slight, it is perhaps more noticeable between Figures 6 and 11.

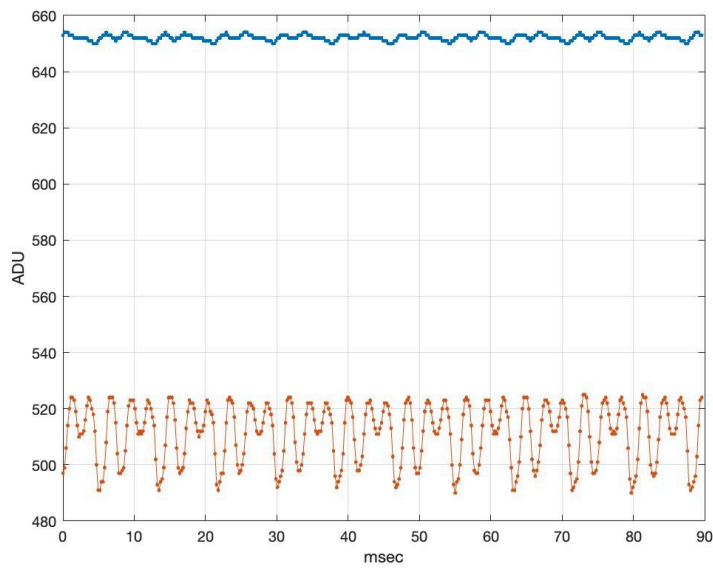


Figure 11

We can also see these changes in the frequency domain in a reduction of the lower frequencies. Figure 12 shows the matching frequency analysis for Figure 10, and Figure 13 shows the matching frequency analysis for Figure 11.

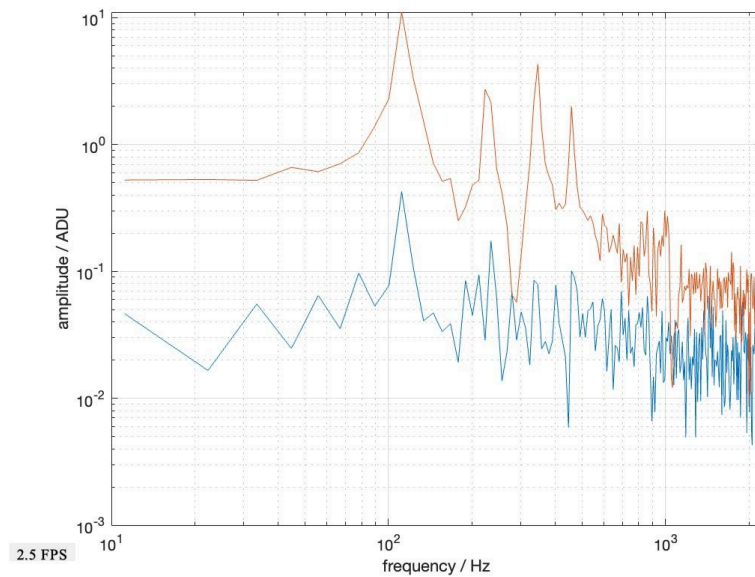


Figure 12

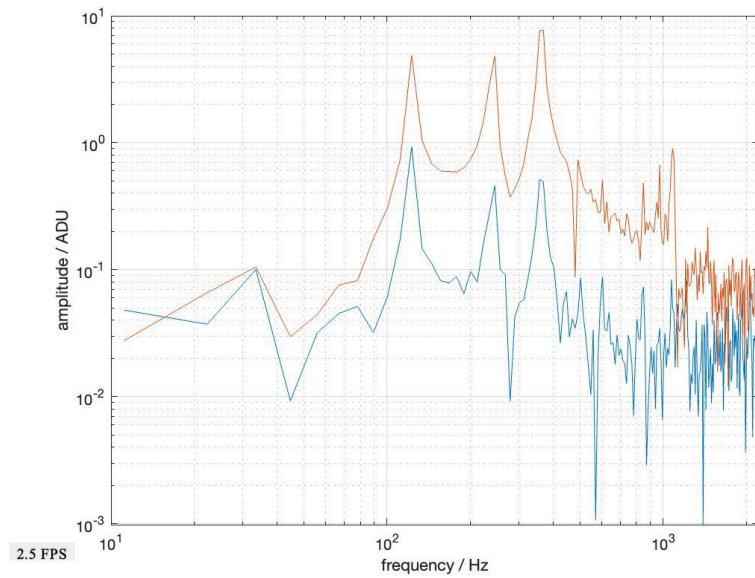


Figure 13

We can see here that, on the left side, the orange line is much lower than in Figure 12, showing a decrease in the presence of low frequency background noise.

And for a better look at the difference between the OpAmp output and the original raw data, Figure 14 shows the data after removing the DC bias and amplifying the raw signal to be visible enough for comparison.

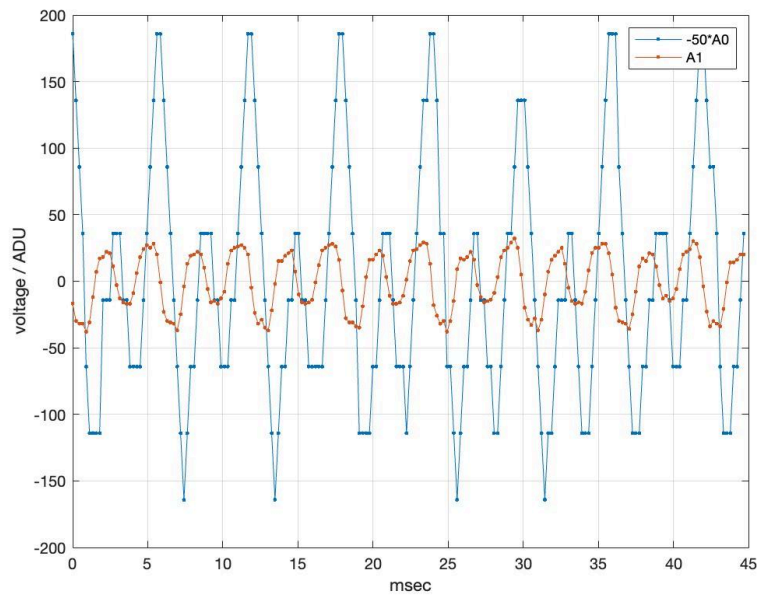


Figure 14

We can see the OpAmp produces a smoother curve and the raw output is more locked to discrete values since it's limited by the ADU of the Arduino and its small amplitude.

Discussion:

A very important question to ask when creating an instrument like this is of course, does it actually work? To answer this, we can run test cases for validation. Feed a known input that has an expected output that should result and see if the actual result matches up with what it should be. For this purpose, I got an online function generator and played a 220 Hz note at a medium volume to test if the scope is working properly. According to this note, the output I get should be around 220 Hz in frequency with a moderate amplitude. The voltage output for this note can be seen in Figure 15 and the frequency output in Figure 16.

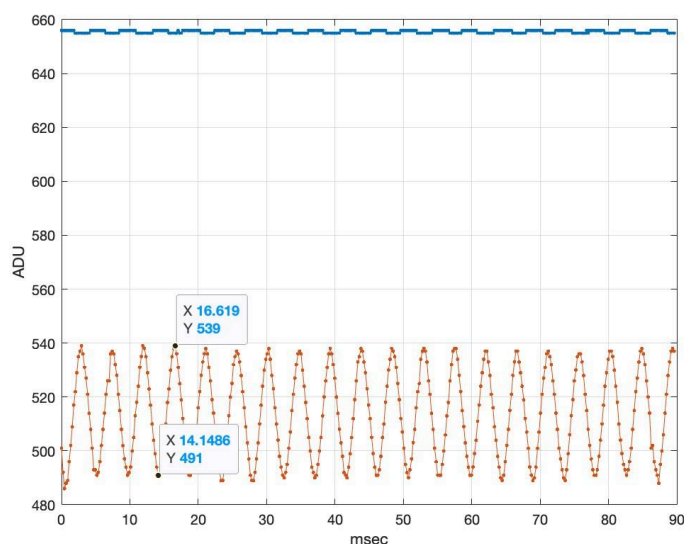


Figure 15

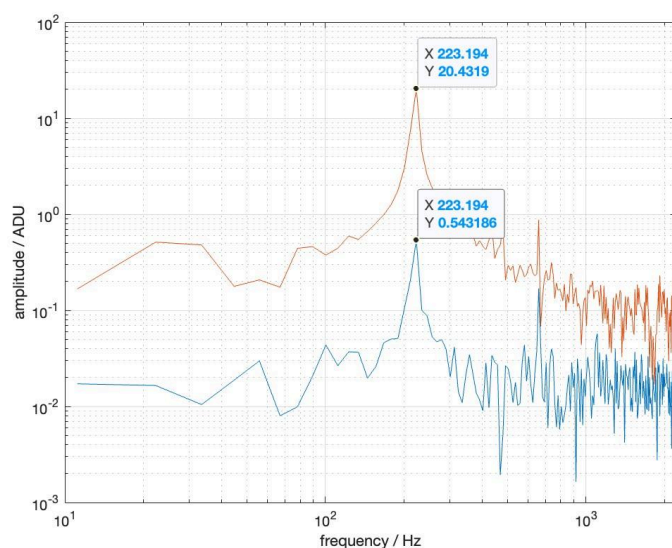


Figure 16

The first graph shows a pretty smooth and steady sine wave at a medium frequency with a medium amplitude. To estimate better, a peak and valley point have been highlighted, showing that the wave spans $539 - 491 = 48$ ADU in total, which means an amplitude of 24 (half). To estimate the frequency from this graph (Fig. 15), we can use the x-component since these points span half a wavelength. So the period would be $T = 2(x_2 - x_1) = 2(16.619 - 14.149) \times 10^{-3} = 0.00494 \text{ seconds}$

Now we can easily estimate the frequency as $f = 1/T \approx 202 \text{ Hz}$

This is not quite right, as we played a 220 Hz note, but this was simply our estimation—not the most accurate as maybe better estimation points could've been taken. However, it is still pretty close. To get a far more accurate account, we can consult the frequency domain, as seen in figure 16. Here, we can see an obvious large spike where the dominant frequency is 223.194—that is *very* close to 220. So this tells us that our oscilloscope is indeed working and giving reasonably accurate results, maybe off by just a small amount. Of course, this doesn't take into account that maybe the signal itself might have been slightly off of exactly 220 Hz due to distortion in the sound over transfer to the microphone, so I think it's safe to say that our oscilloscope works reasonably well.

Conclusion:

I thought this was a particularly interesting lab as it gave us the chance to work with OpAmps and get a feel for how they work and see a little bit of how we can use them to help amplify signals for better results. It also was an interesting exercise in that we have now essentially created an oscilloscope with a reasonable accuracy, which is pretty cool to think about. I thought it made a lot of sense to interface with the Arduino using a command system, and it really showed the power of microcontrollers and computing in that it lets us *significantly* speed up the process of taking data measurements, and then also the power of programming with analyzing that data, especially in the larger quantities that we can get from a microcontroller. So, overall, I felt it was a good learning exercise and an interesting and enjoyable project.

Appendix:

Oscope.ino:

```
/* file: oscopel.ino COMMAND SET
   b -- measure buffer of A0, A1 appended with read time (microsecs), return data in
   binary
   p <pwmValue> analogWrite pwmValue to D5
*/

#define NUMSAMP 400
void measureWaveformsBinary(void);
const int analogOutPin = 5;
uint16_t data[NUMSAMP + 1][2], i;
uint32_t settleTime = 200; // msec
uint32_t t0, t1;

void setup()
{
  Serial.begin(115200/2); // if it works stably, delete /2 for more speed
```



```

pinMode(analogOutPin, OUTPUT);
analogRead(A0); // initialize the ADC system
Serial.println("Ready");
} // setup

void loop()
{
  if (Serial.available())
  {
    switch (Serial.read())
    {
      case 'p': // set pwm value
      {
        int pwmVal = Serial.parseInt();
        analogWrite(analogOutPin, constrain(pwmVal, 0, 255));
        delay(settleTime);
        break;
      }
      case 'b': // measure and write
      {
        measureWaveformsBinary();
        break;
      }
    }
  } // switch
  delay(1);
} // loop

void measureWaveformsBinary()
{
  // Measure as fast as possible A0 and A1 repeating NUMSAMP times
  // return data and time in binary format
  t0 = micros(); // time it to compute sample rate
  for (i = 0; i < NUMSAMP; i++)
  {
    data[i][0] = analogRead(A0);
    data[i][1] = analogRead(A1);
  }
  t1 = micros();
  data[i][0] = (uint16_t)(t1 - t0); // put dt at end of data
  data[i][1] = (uint16_t)((t1 - t0) >> 16); // most significant 2 bytes
  Serial.write((uint8_t*)data, (NUMSAMP + 1) * 2 * sizeof(uint16_t));
} // measureWaveformsBinary

```

Oscope1.m:

```

%% script oscopel.m -- first version of an Arduino oscilloscope display
% 3jan23 BR, Arduino running oscopel.ino, command set = b and p
if ~exist('runBtn','var') || ~isvalid(runBtn) % add a button once
    runBtn = uicontrol('style','radiobutton','string','run','units', ...
        'normalized','position',[.13 .93 .1 .04],'Callback', 'oscopel');
end
if ~exist('ard','var') % initialize arduino
    disp('initializing arduino ...')
    ports = serialportlist; % open the right one
    ard = serialport(ports(end),115200/2,'Timeout',2); % match speed to Arduino
    clear ports; % clean up workspace
    pause(2); % time for Arduino to boot
end
while runBtn.Value
    writeline(ard,'b'); pause(.01);
    bin = read(ard,802,'uint16');
    data = reshape(bin(1:800),2,400)';
    plot(data, '-'); grid;
    xlabel('sample #'); ylabel('voltage/ADU')
end

```

Oscope2.m:

```

%% script oscope2.m -- first version of an Arduino oscilloscope display
% 3jan23 BR, Arduino running oscopel.ino, command set = b and p
addpath('./');
if ~exist('runBtn','var') || ~isvalid(runBtn) % add the button
    runBtn = uicontrol('style','radiobutton','string','run','units', ...
        'normalized','position',[.13 .93 .1 .04],'Callback', 'oscope2');
end
if ~exist('ard','var') % initialize arduino
    disp('initializing arduino ...')
    ports = serialportlist; % works for me, substitute appropriate port for
yours
    ard = serialport(ports(end),115200/2,'Timeout',2);
    clear ports;
    readline(ard);
end
if ~exist('runOnce','var'), runOnce = true; end
while runBtn.Value || runOnce
    writeline(ard,'b'); pause(.01);
    try
        bin = read(ard,802,'uint16');
        dt = bitshift(bin(802),16)+bin(801); % microseconds
        data = reshape(bin(1:800),2,400)';

        figure(3);
    end
end

```

```

    t = linspace(0,dt/1000,400)';
    plot(t, data, '-'); grid;
    xlabel('msec'); ylabel('ADU')
    figure(4);
    sr = length(data)/dt*1e6; % sample rate (#/sec)
    aspectrum(data, sr);
    xlabel('frequency / Hz'); ylabel('amplitude / ADU')
catch
    flush(ard);
    fprintf('*');
end
runOnce = false;
displayfps;
end

```

Aspectrum.m:

```

function [d, x, phase] = aspectrum(v,rate)
% function [s, f, phase] = aspectrum(v,samplerate)
% compute amplitude spectrum of vector v or matrix v of
% column vectors. col vectors of d are spectral components.
% the second argument (optional) is the sample rate (Hz)
% and will plot the spectrum. Phase in degrees.
% f is the frequency axis of the spectrum plot.
% 3-12-92, B.R. deleted dc component from result.
% 13Oct93, B.R. return phase
len = length(v(:,1));
if len == 1, v = v'; len = length(v(:,1)); end
fb = fft(v);
d = 2 * abs(fb) / len;
if nargin == 2
    n = floor(len / 2); % added floor 8may19 BR
    x = (1:n-1) * rate / len;
    d = d(2:n,:); % drop DC
    loglog(x,d)
    xlabel('f (Hz)'); ylabel('amplitude');
    grid;
    if nargin == 3
        phase = angle(fb(2:n,:)) * 180 / pi;
    end
end
end
return;
%% for dc use:
x = (0:n-1)' * rate / 2 / n;
d = d(1:n,:); % added 1-16-92

```

Arduino Controlled DC Power Supply

Objective: Design, build, and test an Arduino controlled LM317 power supply.

Methods:

1. **Hardware Interface:** For this project, a more robust circuit was needed in order to facilitate the controlling of an LM317 linear regulator using the pulse-width modulation (PWM) output from an Arduino Nano. Now, obviously, the first thing to consider is that a PWM signal is an alternating signal going between 0 and 1023 from the digital pin of the Arduino (in this case, D5), and, to reliably control the LM317, a direct current is needed. In order to get a flat signal out of a noisy one, an averaging filter may be applied (like a blurring filter on an image to reduce contrast). This ideally removes the amplitude of the wave-like portions, but still leaves a flat DC voltage. However, when translating this into electrical signals, a physical circuit was needed in order to perform this averaging step. For this purpose, a low-pass filter was created to attenuate the noise and output a flat DC signal from the Arduino (Fig. 1).

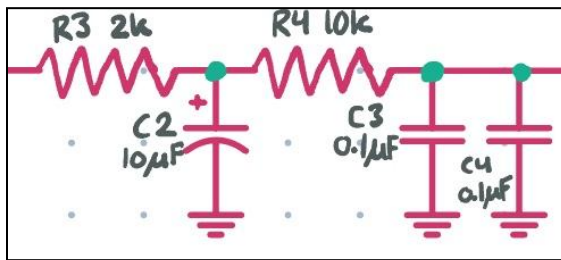


Figure 1

This consisted of two stages of filters, one with a 2k resistor paired with a 10µF capacitor, and the second with a 10k resistor and two 0.1µF capacitors in parallel to make a total 0.2µF capacitance for the second filter. These create a much smoother and flatter DC output voltage as opposed to the erratic raw PWM signal.

Now that the control signal has been prepared, the inner workings of the LM317 integrated circuit must be considered in order to correctly utilize this control signal for our purposes. The equation for the voltage output of the LM317 can be found in its [datasheet](#) as:

$$V_{OUT} = V_{REF} \left(1 + \frac{R2}{R1}\right) + I_{ADJ} R2$$

These quantities correspond to Figure 2:

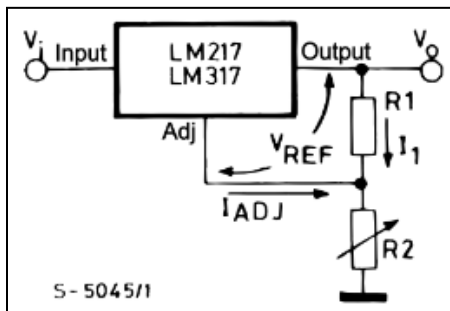


Figure 2

In order to change the output voltage of the device, we can change the voltage across R2. We already do this by manually adjusting the potentiometer for variable resistance, but we can automate this by instead creating a shunt for the current flowing through R2. The output voltage

is V_{REF} more than the voltage across R2, and so by changing the current through R2, we change its voltage since $V_{R2} = I_2 R_2$. Now in this case, we can see that $I_2 = I_{ADJ} + I_1$, but the adjustment current is significantly smaller than I_1 , which means that $I_2 \approx I_1$. So if we change I_1 , we can change V_{R2} and therefore also V_{OUT} automatically. In order to facilitate this process, a current mirror (Fig. 3) was used as a shunt to control the current flow through R2 by siphoning a controlled amount of I_1 away from R2.

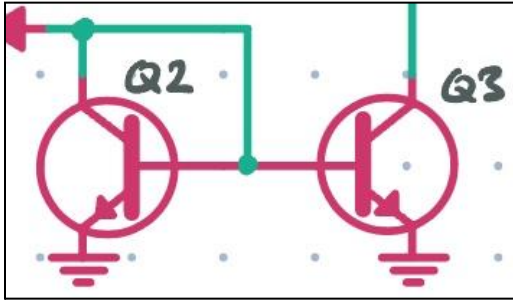


Figure 3

By controlling the reference current on the mirror, we can control how much current is taken from R2, and therefore what the output voltage from the LM317 will be. In order to do this, the logical step is to then control the reference current using the Arduino PWM output from our filters, however, this output does not generate enough current for this usage. A larger current is required, which can be controlled by the PWM signal through the structure of an emitter-follower (Fig. 4).



Figure 4

This emitter follower allows us to control the output current using the LM317 current by exploiting the properties of the active region of a BJT transistor. In this region, the collector current is given by: $I_c = \beta I_b$, and so we can direct the PWM current from the Arduino to the base of the emitter follower transistor and therefore control the output current linearly since the emitter current will be $I_e = I_b + I_c$, and the base current is significantly smaller than the collector. So, by supplying our 12V to the collector, and feeding the emitter in as the I_{REF} of the

current mirror, we can ultimately control how much current is siphoned from the R2 on the LM317 and allow us to change the output voltage through the Arduino PWM output. This chain of components can all be put together as seen in Figure 5.

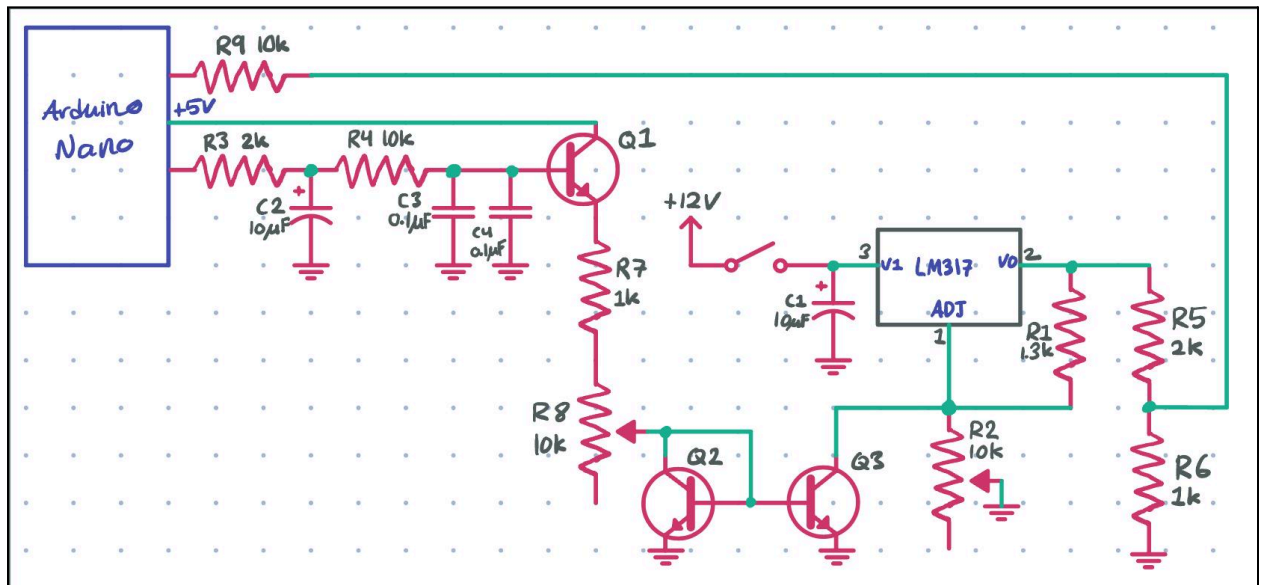


Figure 5

Here we can see how all these pieces fit together and interface with the LM317 (Fig. 5), but there is one more complexity on the emitter-follower—here we have a 10k potentiometer, R8 (Fig. 5). This potentiometer controls the resistance of the emitter-follower resistor, therefore determining the current that will make it to the mirror. This allows for a little room to calibrate the hardware to the desired range.

2. **Software Interface:** For this project, the Arduino requires a slightly more robust command set, involving not only a command to take analog voltage measurements, but also a command to set the PWM value of the Arduino pin for control. In the code for the microcontroller, `oscope1.ino` (see Appendix), it allows for this command as 'p <value>' through the serial connection, where <value> may be any integer from 0 to 255. The code then writes this value to the D5 output pin to provide an “analog” PWM voltage for us to average and use to control the power supply.
3. **Data Interface:** At this point, the Arduino can now be fully controlled for the purposes of outputting a PWM value and reading analog voltage measurements through the serial connection, and so we can interface with it through a USB connection using MATLAB. For this purpose, the `oscope2.m` script (see Appendix) was again used, which repeatedly sends the read command to the Arduino, telling it to read chunks of 400 data points from the analog pins and send it back to MATLAB through the serial port. The Arduino PWM value can also be set through MATLAB using a function, `volts2pwm()`, that was created to translate a given voltage as a parameter into the correct PWM value for the Arduino pin to set the LM317 voltage supply to output that specified voltage.
4. **Calibration:** In order to allow us to enter voltages and get a corresponding Arduino PWM value, the device must be calibrated to the reference voltage and matched with an approximation error

curve which we can use to estimate using the coefficients of a fitted polynomial. To estimate this, we can start by using a linear polynomial and estimating by isolating the linear portion of the voltage curve, as seen in Figure 6, and then fitting a line to it. This was verified by checking the “difference plot”, and then a 3rd order polyfit was also tried and verified.

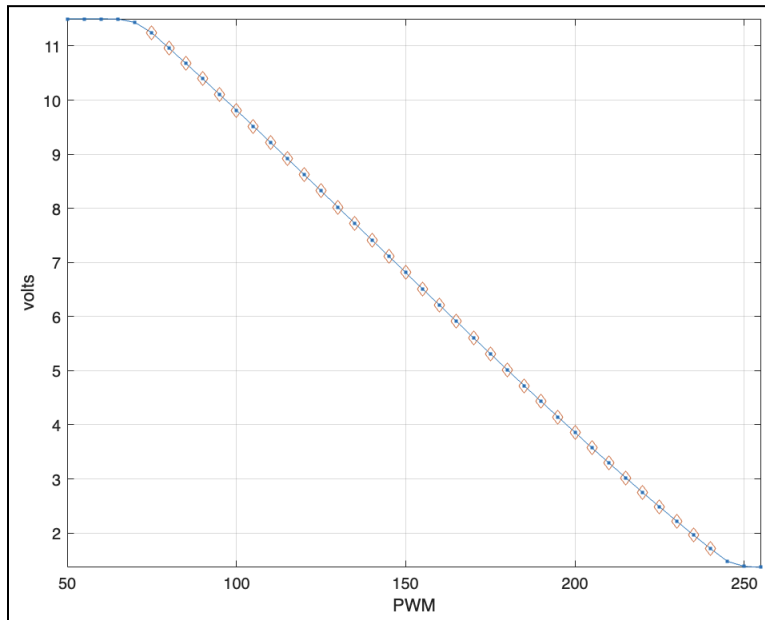


Figure 6

These fitted polynomials act as mapping functions that let us take a value in volts inputted by the user, and turn it into a PWM value from 0-255 that we can give to the Arduino. This lets us easily get the right PWM value to set the power supply to whatever desired voltage in the range.

5. Testing: In order to test and characterize the device, we ran our function to set the voltage at multiple different values and read the output values from the Arduino to directly measure the actual, rather than theoretical, value. We can then compare the actual versus the theoretical in order to calculate the error and standard deviation. Care was taken in this calculation to ensure matching units between measured and theoretical voltages, and to account for the voltage

Results:

1. Before soldering everything together onto the power supply, the circuit was constructed on a solderless breadboard in order to test it and ensure proper operation before finalizing. Figure 7 shows the prototype circuit and Figure 8 shows the measured data curve, which matches what we’re looking for—a linear voltage change as the PWM signal changes.

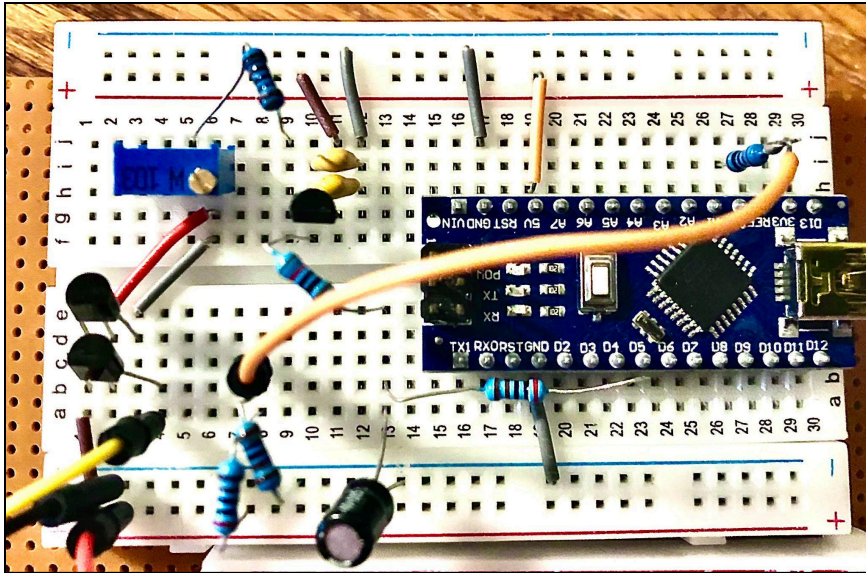


Figure 7

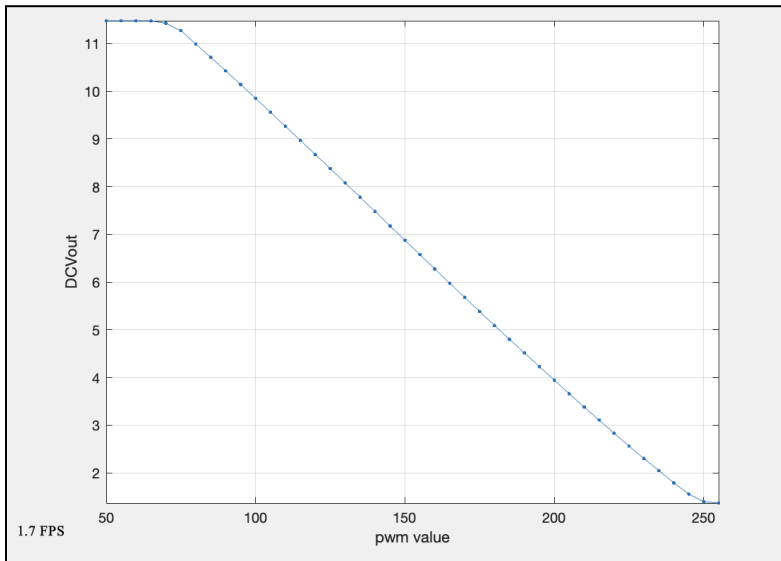


Figure 8

Once it was verified that it was working, the prototype was disassembled and reconstructed alongside the existing power supply circuit, and directly soldered and connected to it. Figure 9 shows the front of the completed circuit, and Figure 10 gives a closeup of the new components.

Figure 9

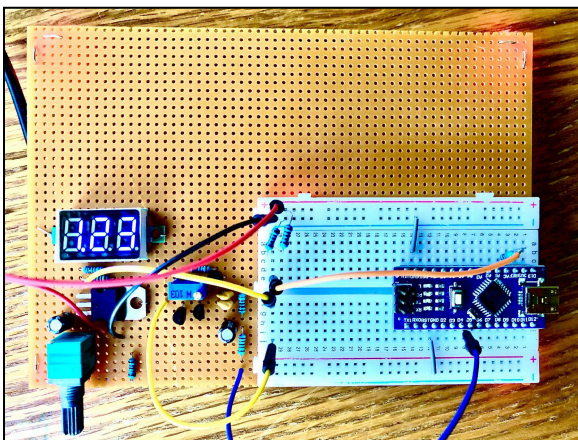
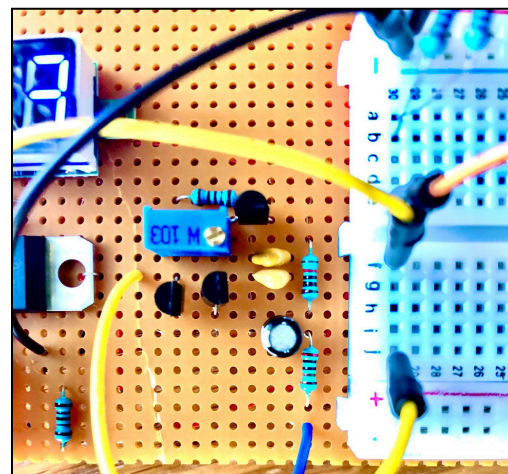


Figure 10



Figures 11 and 12 show the back of the final circuit, with the soldered leads of the components.

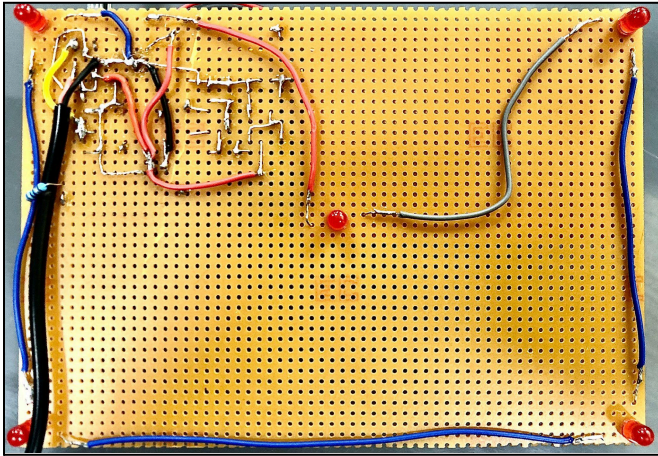


Figure 11

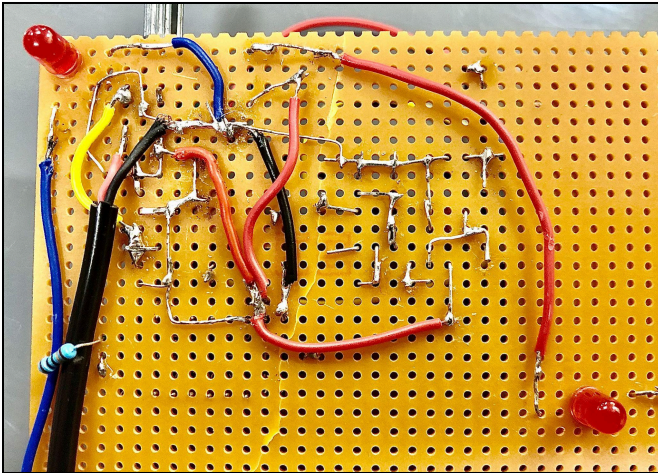


Figure 12

2. Using the software interface, we were able to get an output of the desired pulse-width modulation value from the Arduino using the 'p <value>' command, and this was routed through the filters and into the circuit. The analog voltage reading was also successful in automating our measurements, allowing quick verification and testing of the device.
3. The data interface in MATLAB allowed us to quickly take measurements and set PWM values to verify the circuit and create the `volts2pwm()` function. The calibration step lets us fit a polynomial and get coefficients from it in order to then use in this function to approximate the correct PWM value that corresponds with a given voltage passed as a parameter. Figure 13 shows this function with arbitrary coefficient values.

Figure 13

```
function [pwmVal] = volts2pwm(volts, ard)
% function [pwmVal] = volts2pwm(volts, ard)
% return pwm value corresponding to volts for LM317 arduino ...
% if a serial port is passes as 2nd arg then sends p command
pv = [-0.0438    0.9319  -23.7911  273.4372];
pwmVal = round(polyval(pv, volts));
pwmVal(pwmVal < 0) = 0;
pwmVal(pwmVal > 255) = 255;
if nargin == 2
    writeline(ard, ['p' num2str(pwmVal)]);
end
end
```

4. The result of calibrating the program was that a polynomial was fitted to the data with sufficient accuracy for approximation in reverse mapping from input voltage to PWM value in the `volts2pwm()` function discussed above. The first step in this process was, as shown before in Figure 6, isolating the linear portion of the data, then fitting a line to it. The resulting line can be seen in Figure 14 superimposed over the measured data.

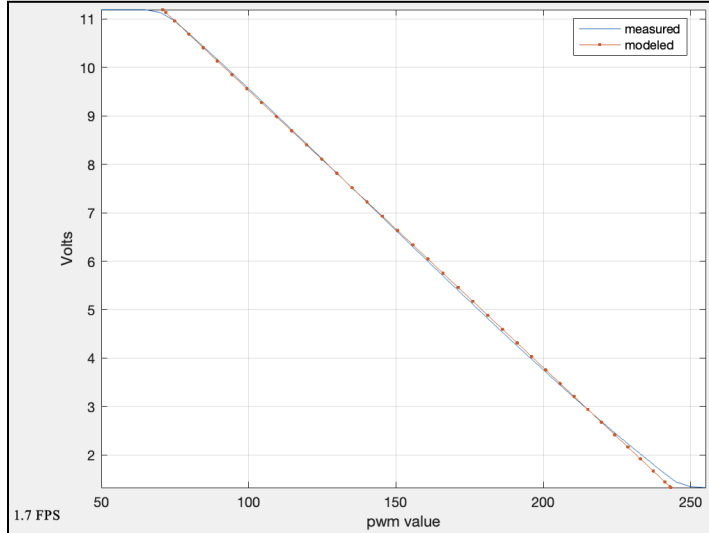


Figure 14

Next, to verify the accuracy of this approximation, the “difference plot” was computed by plotting the difference between the polyfitted and measured points, giving the plot shown in Figure 15. The difference plot for a 3rd degree polyfit is also shown (orange), and we can see that it produces a much more accurate approximation of the output as opposed to the 1st degree linear approximation (blue).

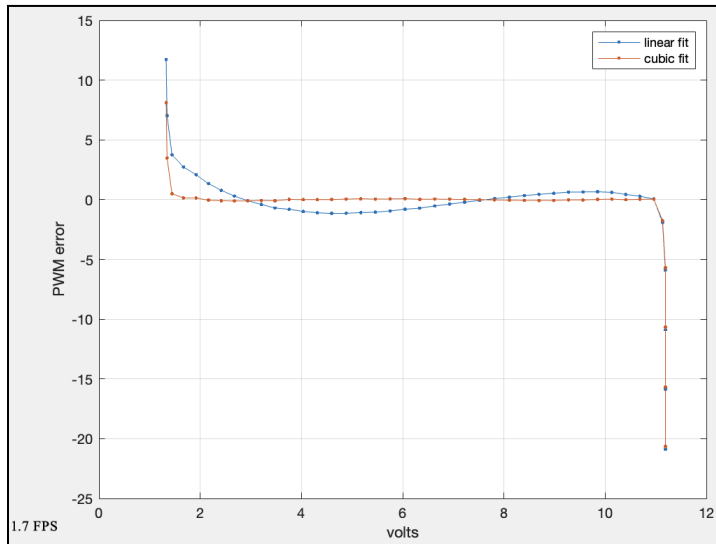


Figure 15

Since the 3rd degree polyfit was much more accurate, it was used in the final function for approximating the PWM value, and the coefficients that were used for this purpose were:

`pv3 = -0.0366    0.7882    -22.5444    275.5021`

These coefficients were then put into the `voltage2pwm()` function, which allowed for relatively accurate mapping from voltages to PWM values based on this specific power supply. This created an ideal setup for testing and error analysis.

5. The result of testing the range of voltage values and checking against actual output values resulted in the graph in Figure 16.

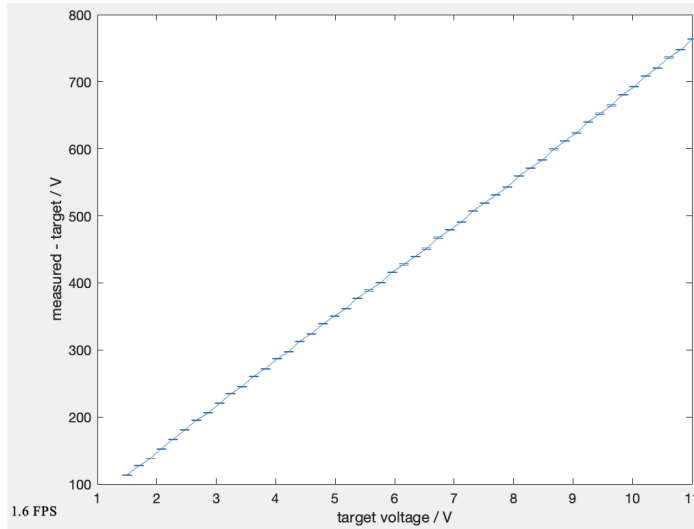


Figure 16

However, the y-axis in this plot is incorrect since it is in units of ADUs not volts. In order to fix this, we simply multiply by a conversion factor of $V_{REF}/1023$. In this case, $V_{REF} = 4.84V$. After this, we also had to account for the voltage divider in the circuit from which we are measuring, using the measured values from R5 and R6: $R5 = 1.99k\Omega$, $R6 = 1.000k\Omega$. The conversion factor for this was $(R5 + R6)/R6$. Multiplying by this finally gave the correct matching units. Once these bugs were fixed, the plot in Figure 17 was given.

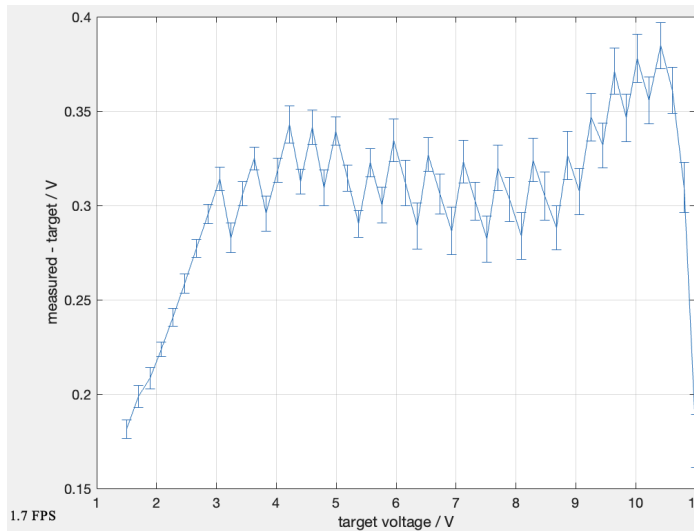


Figure 17

This curve could tell us a number of things, like perhaps that the spikes come from the alternating PWM signal, but a more accurate mapping could likely be achieved through a more complicated

(higher degree) polyfit or more complex algorithm, but this functions with sufficient accuracy for this purpose.

Discussion:

This lab explored many interesting concepts, such as the idea of a shunt with a current mirror to control the LM317 and the usage of a filter to remove the AC component of the Arduino's PWM signal. All-in-all, it was a well-rounded lab that put together many of the concepts we've been learning over the semester about transistors, automation, and op-amps all into one circuit. It shows how all these pieces we've learned about can be put together into a useful system, with each part having a specific purpose in the larger scheme of things, letting us create a more robust, electrically controllable, and programmable voltage supply. It also displays yet again the incredible power of automation in that it lets us use software to control the physical currents to result in different voltages.

Conclusion:

I thought this lab was useful and insightful. To me it was an opportunity to explore a more complex circuit and learn more about each piece and why exactly it works to put all these components together to control the voltage. It allowed me to deepen my understanding of both current mirrors and emitter-followers, and also understand why we need each of them to ultimately achieve our goal. I thought it was pretty incredible to see how each piece had a specific purpose required for automating the power supply. In addition, it was also a good exercise in breaking down slightly more complex circuit diagrams and understanding them on a more abstract level, similar to how we do it in computer science with things like logic circuits and moving to different levels of abstraction as these sort of building blocks (AND, OR, etc... gates, or, in electronics, emitter-followers, current mirrors, etc...) are put together to form more robust systems.

Appendix:

Oscope1.ino:

```
/* file: oscopel.ino COMMAND SET
   b -- measure buffer of A0, A1 appended with read time (microsecs), return data in
   binary
   p <pwmValue> analogWrite pwmValue to D5
*/

#define NUMSAMP 400
void measureWaveformsBinary(void);
const int analogOutPin = 5;
uint16_t data[NUMSAMP + 1][2], i;
uint32_t settleTime = 200; // msec
uint32_t t0, t1;

void setup()
{
  Serial.begin(115200/2); // if it works stably, delete /2 for more speed
  pinMode(analogOutPin, OUTPUT);
  analogRead(A0); // initialize the ADC system
  Serial.println("Ready");
} // setup

void loop()
{
  if (Serial.available())
  {
```

```

switch (Serial.read())
{
    case 'p':    // set pwm value
    {
        int pwmVal = Serial.parseInt();
        analogWrite(analogOutPin, constrain(pwmVal, 0, 255));
        delay(settleTime);
        break;
    }
    case 'b':    // measure and write
    {
        measureWaveformsBinary();
        break;
    }
}
} // switch
delay(1);
} // loop

void measureWaveformsBinary()
{
    // Measure as fast as possible A0 and A1 repeating NUMSAMP times
    // return data and time in binary format
    t0 = micros();    // time it to compute sample rate
    for (i = 0; i < NUMSAMP; i++)
    {
        data[i][0] = analogRead(A0);
        data[i][1] = analogRead(A1);
    }
    t1 = micros();
    data[i][0] = (uint16_t)(t1 - t0); // put dt at end of data
    data[i][1] = (uint16_t)((t1 - t0) >> 16); // most significant 2 bytes
    Serial.write((uint8_t*)data, (NUMSAMP + 1) * 2 * sizeof(uint16_t));
} // measureWaveformsBinary

```

Oscope2.m:

```

%% script oscope2.m -- first version of an Arduino oscilloscope display
% 3jan23 BR, Arduino running oscopel.ino, command set = b and p
addpath('./');
if ~exist('runBtn','var') || ~isvalid(runBtn) % add the button
    runBtn = uicontrol('style','radiobutton','string','run','units',...
        'normalized','position',[.13 .93 .1 .04],'Callback','oscope2');
end
if ~exist('ard','var') % initialize arduino
    disp('initializing arduino ...')
    ports = serialportlist; % works for me, substitute appropriate port for
yours
    ard = serialport(ports(end),115200/2,'Timeout',2);
    clear ports;
    readline(ard);
end
if ~exist('runOnce','var'), runOnce = true; end
while runBtn.Value || runOnce
    writeline(ard,'b'); pause(.01);
    try
        bin = read(ard,802,'uint16');
        dt = bitshift(bin(802),16)+bin(801); % microseconds
        data = reshape(bin(1:800),2,400)';

        figure(3);
    end
end

```



```

    t = linspace(0,dt/1000,400)';
    plot(t, data, '.-'); grid;
    xlabel('msec'); ylabel('ADU')
    figure(4);
    sr = length(data)/dt*1e6; % sample rate (#/sec)
    spectrum(data, sr);
    xlabel('frequency / Hz'); ylabel('amplitude / ADU')
catch
    flush(ard);
    fprintf('*');
end
runOnce = false;
displayfps;
end

```

Arduino Controlled Function Generator

Objective: Design, build, and test an Arduino controlled function generator.

Methods:

1. **Hardware Interface:** For this project, the majority of the hardware needed was already present from the automation of the LM317 linear regulator. The primary and most prominent addition since that part was the function generator itself. This was built from a kit with a PCB and the proper components. The instructions and specification may be found on this website here: <https://helpfulcolin.com/xr2206-function-generator-kit-improved-instructions/>. The components were soldered into their places as dictated by the instructions, and two small modifications were made to reverse the direction of increase on the amplitude potentiometer. The other addition was to, instead of attaching the provided power socket, solder a 1M Ω resistor across the terminals and this was connected to the LM317 power supply, though, rather than receiving the output of the linear regulator, it received the 11-12V straight from the switch. Beyond this, the setup for the automation of this device was simply to solder a wire to R6 (Fig. 1) and use that to shunt current for control.

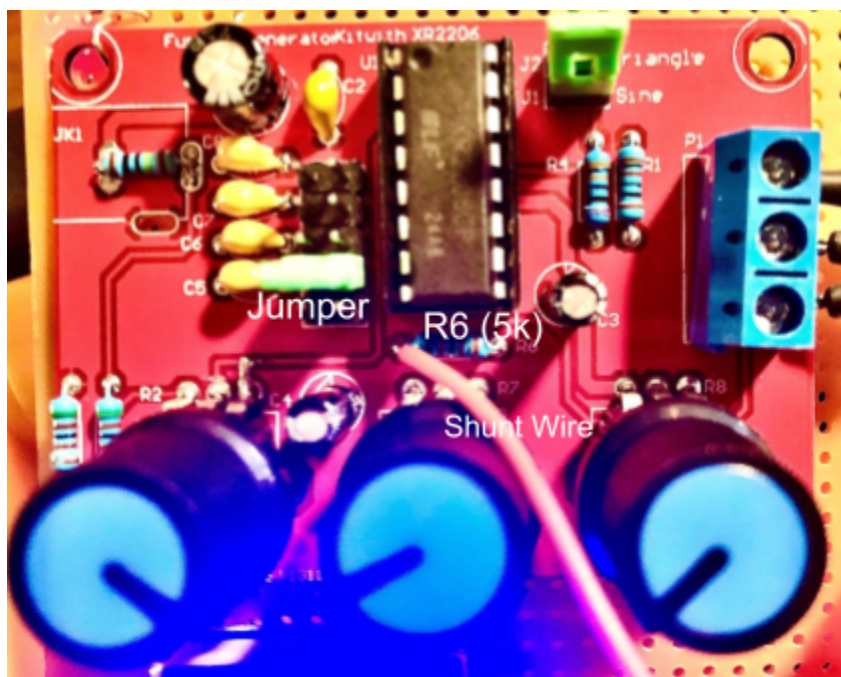


Figure 1: Function generator with modifications and shunt wire.

The function generator uses the XR2206 Monolithic Function Generator integrated circuit, whose datasheet can be found here: <https://cdn.sparkfun.com/assets/8/a/b/3/9/XR2206.pdf>. The datasheet tells us a bit more about the circuit, which we need to further analyze in order to determine our approach for automation. In this case, the basic test circuit is as so (Figure 2):

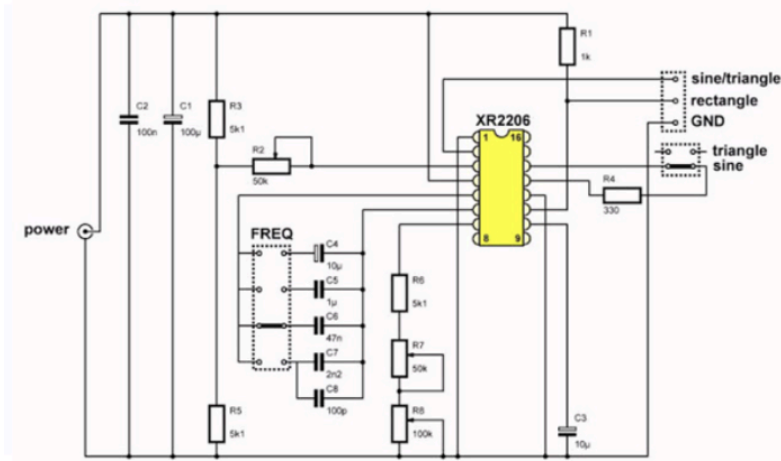


Figure 2: Basic Test Circuit for XR2206 IC

This schematic is very similar to our PCB, With the different capacitors to specify the frequency range and the pullup resistor to limit the power to the IC to safe conditions. Since these are such similar circuits, it then makes sense that the datasheet's specifications shall be applicable to our specific PCB as well. This is useful because the datasheet outlines how the frequency is determined and lets us ponder the different ways we might actually control it using a PWM signal. From this, we find two interesting equations:

$$f_0 = \frac{1}{RC} \text{ Hz and } f = \frac{320I_T(\text{mA})}{C(\mu\text{F})} \text{ Hz}$$

This gives us two different ways to control the frequency—one is the typical method using the first equation that it already uses with the two frequency potentiometers. This controls the R in the equation, changing the frequency by changing the resistance. However, the more applicable one in our case for automation is actually the second one. We can see in this equation that the frequency is directly proportional to the timing current, I_T . If we recall before, with the LM317 automation, we faced a similar problem, and solved it using a current mirror linked through an emitter follower and some filters to the PWM Arduino signal. In fact, the situation is so much the same that we can actually use that same circuit here! We've already built it on the board, so why not make use of it? In order to verify this, we would test the equation by measuring the frequency range and checking against our theoretical expectations, but my voltmeter did not have adequate accuracy to measure in Hz, only kHz, and so I could not perform this measurement properly. However, from the results given in the lab file, we could see that the 2π normally present in the corner frequency equation $f_c = \frac{1}{2\pi fC}$ has been essentially hardcoded into the IC already, so we don't have to worry about it, and can simply consider the equation given by the datasheet,

$$f_0 = \frac{1}{RC}.$$

Now, in order to get the function generator ready for automation, all we have to do is connect it to the circuit we already created for the LM317. In order to do this, we connect a wire to the node with pin 7 of the IC and the resistor R6 (Fig. 1). If we placed the wire after the resistor, the maximum current we could shunt from it would be around $3V / 5k = 0.6\text{mA}$, whereas bypassing it gives us control up to the full 1mA of current. A closer view of this connection can be seen in Figure 3.



Figure 3: Shunt wire soldered onto R6 on the PCB.

This was then connected to the rest of the current mirror, emitter-follower, etc, and ultimately to the Arduino PWM. The schematic for this setup may be seen in Figure 4.

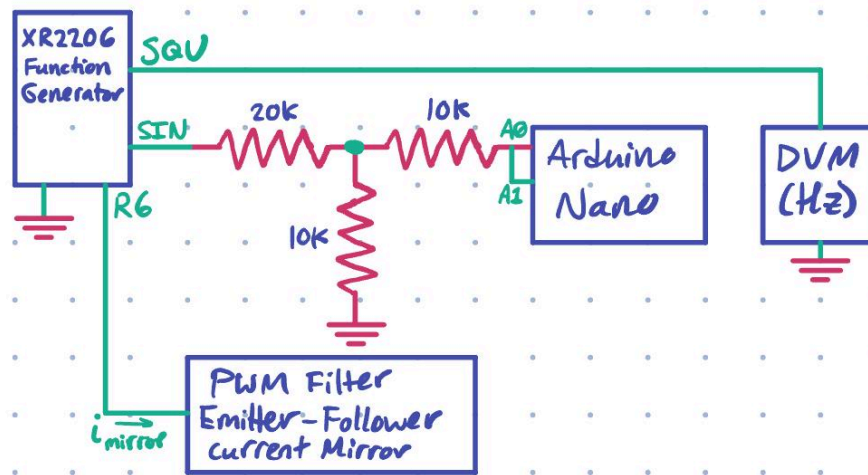


Figure 4: Schematic for control setup.

We can see in this schematic how the current mirror will shunt current from the generator, effectively changing I_T , which will, in turn, change the frequency output.

2. Software Interface: For this project, the Arduino ran the same program from before, with a command to read data, and a command to set the PWM value. Both of these are obviously important for this specific task, the PWM command for altering the shunt current and setting the output frequency, and the reading command to actually see the output voltage waveform and to verify that this is really working.
3. Data Interface: At this point, the Arduino can now be fully controlled for the purposes of outputting a PWM value and reading analog voltage measurements through the serial connection, and so we can interface with it through a USB connection using MATLAB. However, this time, we are dealing with an alternating signal that suffers from phase shift between the A0 and A1 measurements on the Arduino pins. In order to fix this, a Fourier deskewing was applied, which effectively lined up the phases of these two measurements, giving us a cleaner output to analyze.

In order to do this, a new program was created from oscope2.m, improving and iterating upon it. This new program was saved as oscope3.m (see Appendix), and incorporated the Fourier deskewing for better outputs in alternating voltages.

4. **Calibration:** In order to verify and set the frequency automation to the desired frequency range, a range of PWM values were set and the output frequencies were measured in order to get an idea of the current state of the device and what the current frequency range was. Once this was established, the jumper on the PCB (Fig. 1) was moved to the center position instead, raising the frequency range. This was then measured again and results were analyzed. Following this, an alternate capacitor array was connected to the circuit instead, in order to specify a certain desired frequency range.

After this, similar to the LM317 lab, we created a function, freq2pwm() (Fig. 5) to convert a frequency value to a PWM value for the Arduino to make setting the frequency much easier for us. In a similar manner as before, multiple polyfits were performed and then the best was used as an approximation tool to perform this mapping.

```
function [pwmVal] = freq2pwm(freq, ard)
% function [pwmVal] = volts2pwm(freq, ard)
% return pwm value corresponding to freq (Hz) for XR2206 arduino control
% using the identical current mirror circuit built for the LM317 and
% volts2pwm function
% if a serial port is passes as 2nd arg then sends p command

pf = [ -26.9427  145.0685 -308.5329  331.4820 -196.6616  193.5220 -99.4645];
pwmVal = round(polyval(pf, freq / 1e3)) + 150;
pwmVal(pwmVal < 0) = 0;
pwmVal(pwmVal > 255) = 255;

if nargin == 2, writeline(ard,['p' num2str(pwmVal)]); end
end
```

Figure 5: Frequency to PWM conversion function in MATLAB.

5. **Verification:** In order to test and characterize the device, we ran our function to set the voltage at multiple different values and read the output values from the Arduino to directly measure the actual, rather than theoretical, value. We can then compare the actual versus the theoretical in order to calculate the RMS error.

Results:

1. The final board can be seen in Figure 6 with the function generator connected to the arduino through the current mirror circuit:

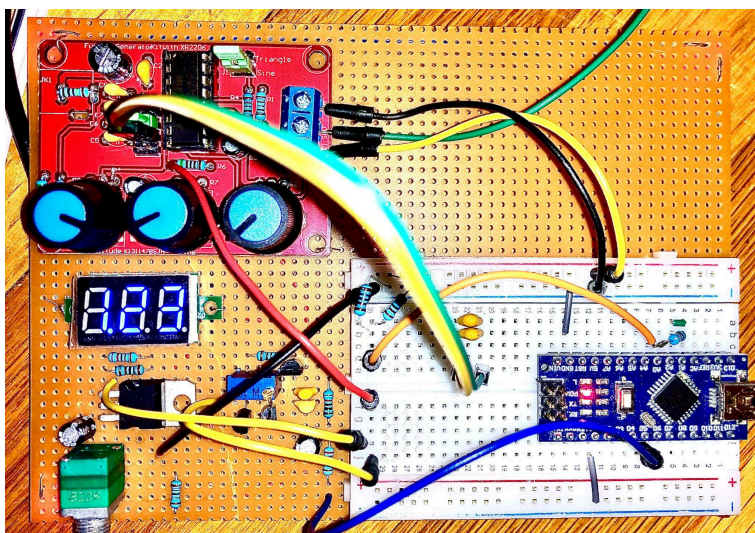


Figure 6: Finished Board.

2. Using the software interface, we were able to get an output of the desired pulse-width modulation value from the Arduino using the 'p <value>' command, and this was routed through the filters and into the circuit. The analog voltage reading was also successful in automating our measurements, allowing quick verification and testing of the output frequencies.
3. The data interface allowed us to get rid of problems with phase differences using Fourier deskewing to get a clean output for both input channels. In this way, we were able to see the output sine wave and get an idea for the amplitude and frequency at any given time.
4. For the first range of frequencies that were measured, the jumper was in the second position from bottom, so it gave a lower range of frequencies. This range can be seen in Figure 7:

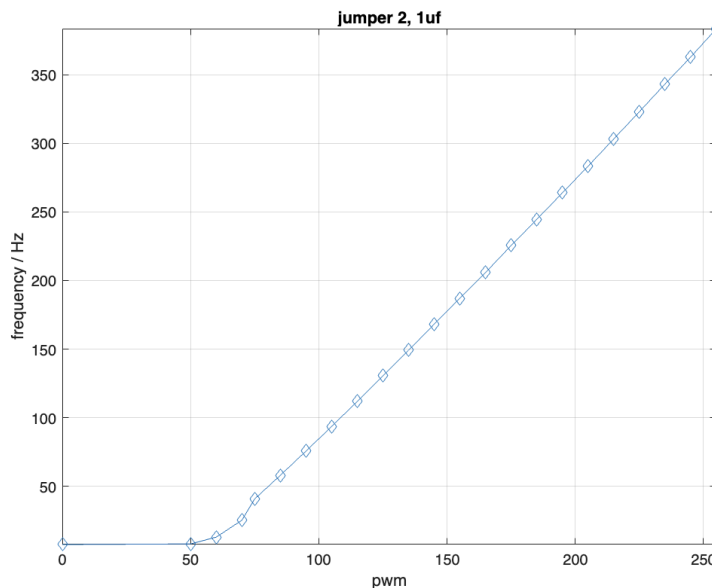


Figure 7: Graph of PWM value versus output frequency from the function generator.

In this case, the first 3-4 data points may be ignored since oscscope3 is unable to measure some lower frequencies like this and my digital multimeter did not have enough precision to measure it either. For the rest of the data, however, we can see a clear linear trend, which is a good sign, showing us that our PWM value is directly affecting the output frequency as it changes.

Following this, we performed the same measurements, cycling through the full PWM range, but with the jumper in the center position, raising the frequency range. This produced perhaps the most interesting results as can be seen in Figure 8:

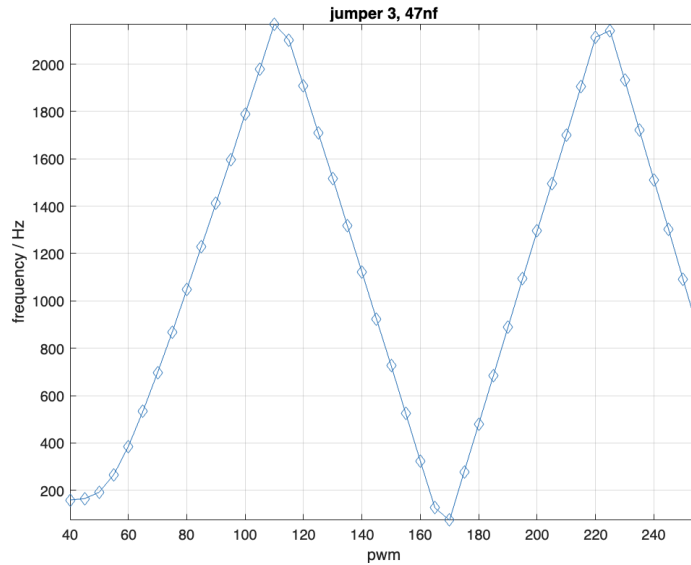


Figure 8: Graph of PWM versus frequency with higher frequency range.

This graph produces this interesting wonky line that reflects off the top, then off the bottom, then off the top again. This is due to aliasing—the higher frequencies exceed the Nyquist frequency, which is half the sample rate. Our Nyquist frequency is around 2232 with the Arduino, and so when we exceed that, we can see that the frequency sort of reflects back downwards instead to the sample rate minus the frequency rather than the actual frequency value. This happens multiple times as it continues to pass multiples of the Nyquist frequency, and so we can see it bouncing multiple times. To prevent this from happening, we will instead specify a frequency range of about 30Hz to 2.2kHz to prevent it from passing the Nyquist frequency. We do this by connecting two 0.1 μ F capacitors instead of the normal jumper to the second place from the bottom. This created the following output curve (Figure 9):

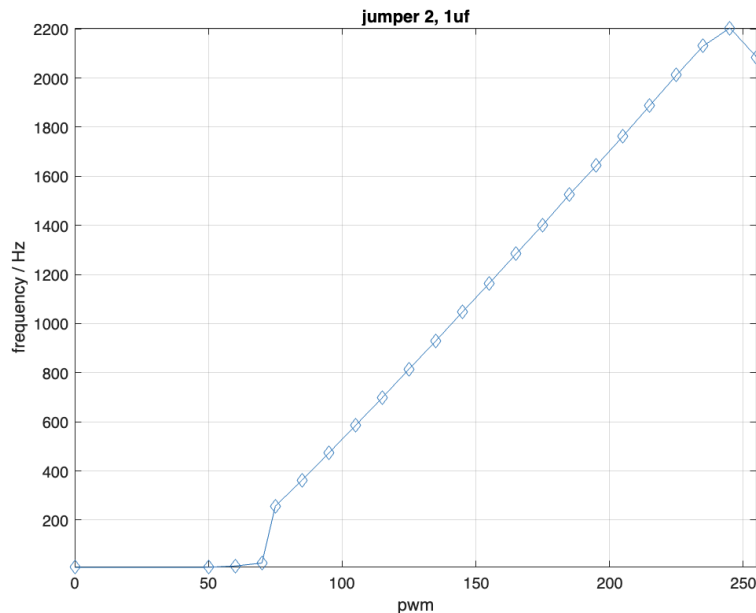


Figure 9: Specified frequency range output.

Again, discount the first 4 data points, but we can see there is still a little bit of aliasing at the end, so we also connect the jumper in the middle position in addition to the parallel capacitors we added. This produces a much smoother curve with our desired frequency range (Fig. 10):

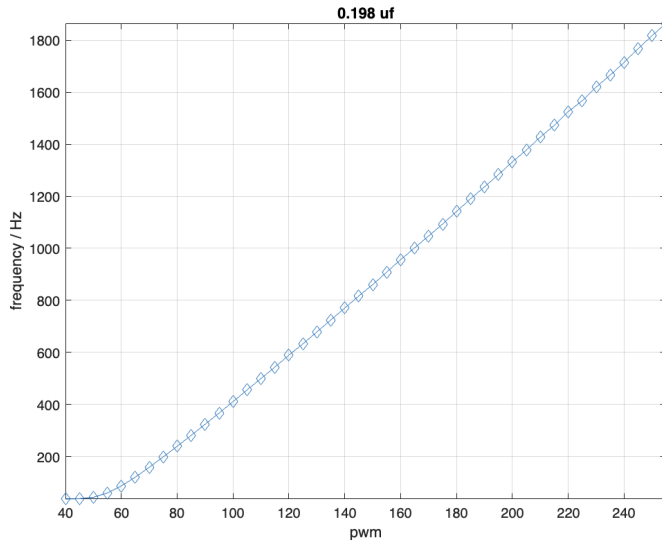


Figure 10: Smoother output curve at desired range.

This is a much nicer output that will fit our purposes quite nicely.

Now, in order to fully calibrate the device, we want to create a function to map an input frequency value into a PWM value for the Arduino. In order to do this, we try mapping different orders of polynomials to the data and check the error they have from the actual measured values. This is the error mapping (Fig. 11).

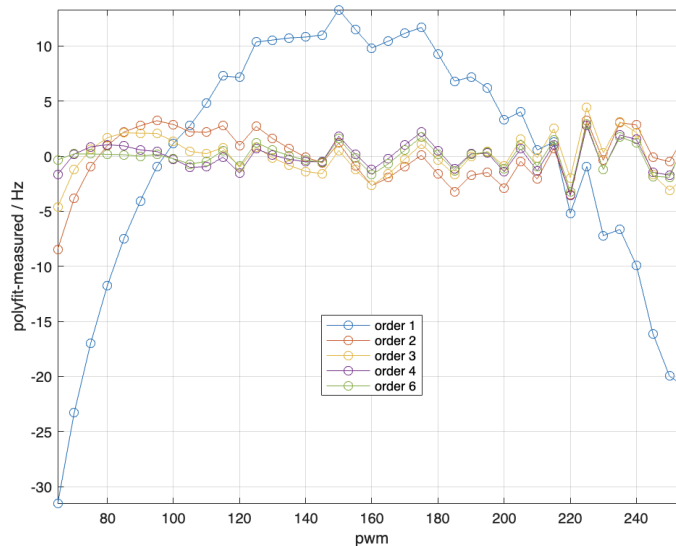


Figure 11: Error graphs for different order of polynomials.

After this, the best polynomial fit was selected and its coefficients were placed into the `freq2pwm` function for more accuracy after a bit of balancing.

- To validate and verify the system, the freq2pwm function was ran with 50 frequencies and plotted against the actual output value to test its accuracy (Fig. 12):

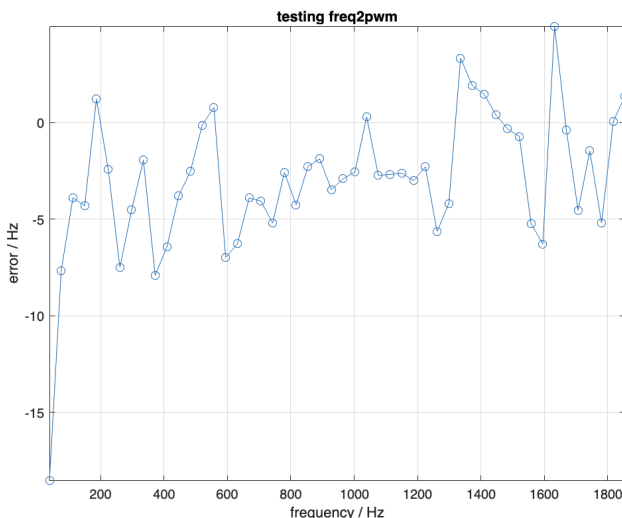


Figure 12: Graph of error for testing.

The majority of the error is not too large, being within about 2-3% for the most frequencies.

Discussion:

This lab allowed us to explore AC function generators more in-depth and learn about complexities that arise with things like sample frequency and aliasing when we hit the Nyquist frequency. I actually had learned about this before in Digital Image Processing, and so it was really interesting to see another application of this in electronics for voltage analysis. It makes a lot of sense since these are AC signals with frequencies that can be measured, and, of course, whenever we are measuring something at a certain rate, we reach limits when the thing we are measuring is higher frequency than our measuring rate.

Conclusion:

I thought this lab was really interesting and showed another application of this system of current mirrors and such to control systems with an Arduino. This also showed kind of the brilliance of not “reinventing the wheel”, so to speak. We already had all of the necessary hardware in place to control the system, and so all we had to do was reuse it for this purpose. So overall I thought it was an interesting lab that gave insight into the frequency domain of voltages and also more into how we can control these integrated circuits through the current mirrors connected to the Arduino.

Appendix:

Oscope1.ino:

```
/* file: oscope1.ino COMMAND SET
   b -- measure buffer of A0, A1 appended with read time (microsecs), return data in
   binary
   p <pwmValue> analogWrite pwmValue to D5
*/

#define NUMSAMP 400
void measureWaveformsBinary(void);
const int analogOutPin = 5;
uint16_t data[NUMSAMP + 1][2], i;
```

```

uint32_t settleTime = 200; // msec
uint32_t t0, t1;

void setup()
{
  Serial.begin(115200/2); // if it works stably, delete /2 for more speed
  pinMode(analogOutPin, OUTPUT);
  analogRead(A0); // initialize the ADC system
  Serial.println("Ready");
} // setup

void loop()
{
  if (Serial.available())
  {
    switch (Serial.read())
    {
      case 'p': // set pwm value
      {
        int pwmVal = Serial.parseInt();
        analogWrite(analogOutPin, constrain(pwmVal, 0, 255));
        delay(settleTime);
        break;
      }
      case 'b': // measure and write
      {
        measureWaveformsBinary();
        break;
      }
    }
  } // switch
  delay(1);
} // loop

void measureWaveformsBinary()
{
  // Measure as fast as possible A0 and A1 repeating NUMSAMP times
  // return data and time in binary format
  t0 = micros(); // time it to compute sample rate
  for (i = 0; i < NUMSAMP; i++)
  {
    data[i][0] = analogRead(A0);
    data[i][1] = analogRead(A1);
  }
  t1 = micros();
  data[i][0] = (uint16_t)(t1 - t0); // put dt at end of data
  data[i][1] = (uint16_t)((t1 - t0) >> 16); // most significant 2 bytes
  Serial.write((uint8_t*)data, (NUMSAMP + 1) * 2 * sizeof(uint16_t));
} // measureWaveformsBinary

```

Oscope3.m:

```

%% oscope3.m -- adds title with freq, amplitudes, phase diff,
% and Fourier deskew
if ~exist('runBtn','var') || ~isvalid(runBtn)
  runBtn = uicontrol('style','radiobutton','string','run', ...
    'units','normalized','position',[.13 .93 .1 .04]);
  runBtn.Callback = 'oscope3';
end
if ~exist('ard','var') % initialize arduino
  disp('initializing arduino ...')
  ports = serialportlist;

```

```

ard = serialport(ports{end}, 115200/2);
ard.Timeout = 2;
clear ports;
pause(2); % time to boot
end
if ~exist('runOnce','var'), runOnce = true; end
while runBtn.Value || runOnce
    writeline(ard,'b');
    try
        bin = read(ard,802,'uint16');
        dt = bitshift(bin(802),16)+bin(801); % microseconds
        rawdata = reshape(bin(1:800),2,400)';
        % set 2nd arg of trim to ADU value with steepest slope
        try
            [rdata, npds] = trim(rawdata, 325)
        catch
            fprintf("Trim Failed");
        end
        t = linspace(0,dt/1000,400)';
        t = t(1:length(rdata));
        sr = length(rawdata)/dt*1e6; % sample rate (#/sec)
        freq = sr/(length(rdata)/npds); % Hz, 1/sec

        data = arddes skew(rdata, 'fourier');
        %data = arddes skew(rdata, 'spline');

        fd = fft(data); % data is trimmed to npds
        ampl = abs(fd(npds+1,:)) / length(data) * 2;
        theta = rad2deg(angle(fd(npds+1,:)));
        plot(t, data, '-');
        xlabel('msec'); ylabel('ADU'); grid on;
        title(sprintf('%.2fHz V1=%.2f V2 = %.2f deg=%.2f', ...
            freq, ampl, diff(theta)), 'FontSize',12);

        displayfps;
    catch
        flush(ard);
        fprintf('*');
        break;
    end
    runOnce = false;
end

```